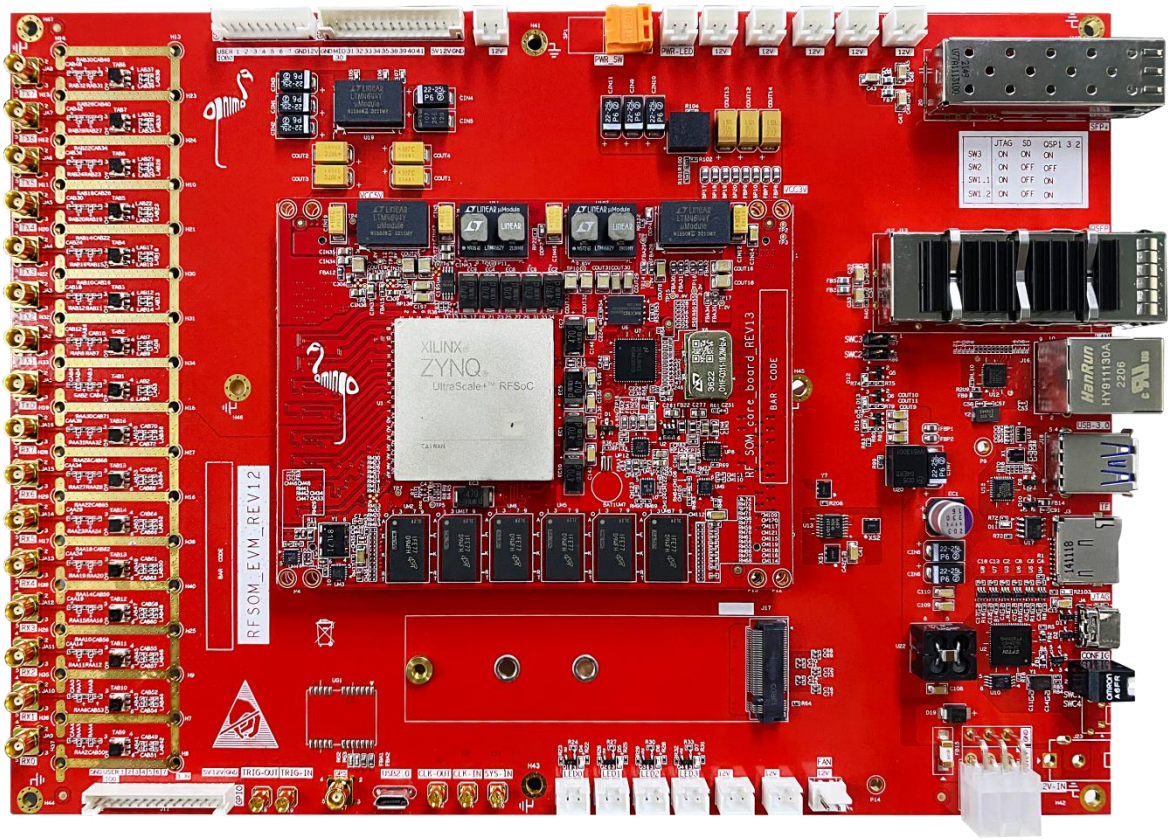
RFEVM 开发平台

Linux 基础教程

Rev. 1.0



修订记录

版本	修订日期	修订内容
1.0	2023 年 7 月 25 日	

目录

修订记录 **2**

1、Petalinux 的安装 **5**

1.1 Petalinux 简介 5

1.2 下载 Petalinux 安装包 5

1.3 安装 Petalinux 6

1.4 设置 Petalinux 环境变量 8

2、使用 Petalinux 定制 Linux 系统 **9**

2.1 Petalinux 建立工程 9

2.2 配置内核 11

2.3 配置根文件系统 12

2.4 配置设备树 12

2.5 生成启动镜像 15

2.4 启动系统 16

2.5 保存一个工程为模板工程 17

3、配置系统默认 IP **17**

3.1 创建 meta-user layer 17

3.2 编译工程 19

4、添加开机自启动脚本及用户文件 **20**

4.1 自启动脚本创建 20

4.2 编译工程 22

4.3 自启动测试 23

5、使用 Vitis 开发 Linux 应用 **23**

5.1 Vitis 安装 24

5.2 创建基于 Vitis 的 Linux 平台工程 24

5.3 创建 Linux 应用工程 29

5.4 在开发板运行程序 33

5.5 使用 TCF Agent 方式运行 36

6、SD 卡根文件系统 **39**

6.1 Petalinux 配置 40

6.2 SD 卡分区制作 40

6.3 SD 卡镜像文件拷贝 44

7、Linux 下 NVME SSD 操作 45

7.1 petalinux 配置 45

7.2 读写测速 47

8、USB 的使用 48

8.1 USB 接口简介 48

8.2 USB 传输 49

8.3 USB 总线驱动 49

8.4 USB 设备驱动 50

8.5 URB 请求块 53

8.6 USB 测试 55

9、以太网的使用 56

9.1 linux 驱动框架 57

9.2 网络驱动实验测试 63

9.3 外网连接测试 67

9.4 以太网网速测试 67

1、Petalinux 的安装

本章将带大家来安装 Petalinux 开发工具，需要注意的是与 Vivado 工具不同的是我们不是将 petalinux 工具安装在 Windows 系统下，而是安装在 Ubuntu 操作系统中，所以在此之前确保大家已经在虚拟机中安装了 Ubuntu 18.04.4 64 位操作系统，这也是官方推荐的版本，那么对于其他版本可能会在安装和使用过程中出现莫名其妙的错误，所以这里不推荐大家使用其他版本。

1.1 Petalinux 简介

Petalinux 工具是 Xilinx 公司推出的嵌入式 Linux 开发套件，包括了 u-boot、Linux Kernel、device-tree、rootfs 等源码和库，以及 Yocto recipes，可以让客户很方便的生成、配置、编译及自定义 Linux 系统。Petalinux 支持 Versal 、Zynq UltraScale+ MPSoC、Zynq-7000 SoC 以及 MicroBlaze，可与 Xilinx 硬件设计工具 Vivado 协同工作，大大简化了 Linux 系统的开发工作。

具体的介绍可访问 Petalinux 工具网站：

https://china.xilinx.com/products/design-tools/embedded-software/petalinux-sdk.html

1.2 下载 Petalinux 安装包

可以到 Xilinx 的官方网站进行下载：

https://china.xilinx.com/support/download/index.html/content/xilinx/zh/downloadNav/embedded-design-tools/archive.htmlhttps://china.xilinx.com/support/download/index.html/content/xilinx/zh/downloadNav/embedded-design-tools.html

如下图所示（注：需要注册账号才能下载）：



由于 Petalinux 的版本要求与 Vivado 的版本一致，而我们使用的 Vivado 选用的是 2021.2 版本，所以这里应选择 2021.2 版本。点击“2021.2”，弹出该版本页面后，找到 petalinux 的安装文件，如下图所示。



然后点击“Petalinux 2021.2 安装程序”进行下载，当点击之后会出现如下的一个页面，这里需要登录账户才能进行下载；如果之前注册过账户直接登录之后就可以下载了，如果还没有账户，则可以点击下方的“创建账号”字样进行注册，关于注册过程就不细说了，按照提供的步骤一步一步做就可以了

下载完成之后就获得了 petalinux 的安装文件 petalinux-v2021.2-final-installer。

1.3 安装 Petalinux

首先将 petalinux 安装包文件 petalinux-v2021.2-final-installer.run 拷贝到 Ubuntu 系统的家目录中，打开终端可以看到 petalinux 的安装文件

petalinux-v2021.2-final-installer.run 已存在，可直接访问，如下图所示：

```
sdr@sdr:~$ ls petalinux-v2021.2-final-installer.run
petalinux-v2021.2-final-installer.run
sdr@sdr:~$
sdr@sdr:~$
sdr@sdr:~$
```

其次安装 petaLinux 所需要的库，在安装之前需要确定联网：

sudo apt update

sudo apt-get install gawk wget git-core diffstat unzip texinfo gcc-multilib build-essential chrpath socat cpio python python3 python3-pip python3-pexpect xz-utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa libsdl1.2-dev pylint3 xterm pax screen

zlib1g-dev texinfo libtool autoconf libselinux1 bison flex libssl-dev libncurses5-dev gcc g++ net-tools zlib1g:i386

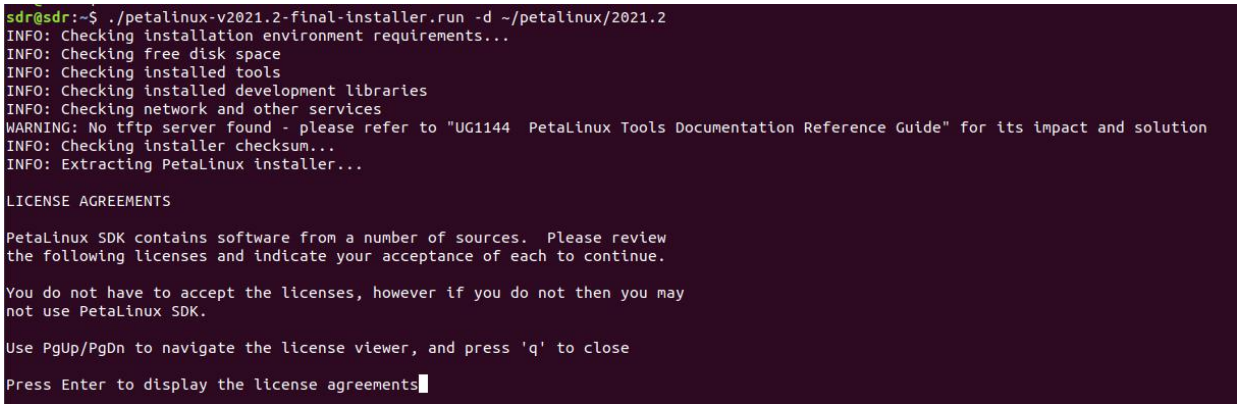
安装 Petalinux 就要考虑安装位置了，对于 Petalinux 这种体积庞大的工具，我们需要单独建立一个文件夹。在家目录下新建专门存放 Petalinux 的文件夹，如~/petalinux/2021.2，在终端输入以下命令即可：

mkdir -p ~/petalinux/2021.2

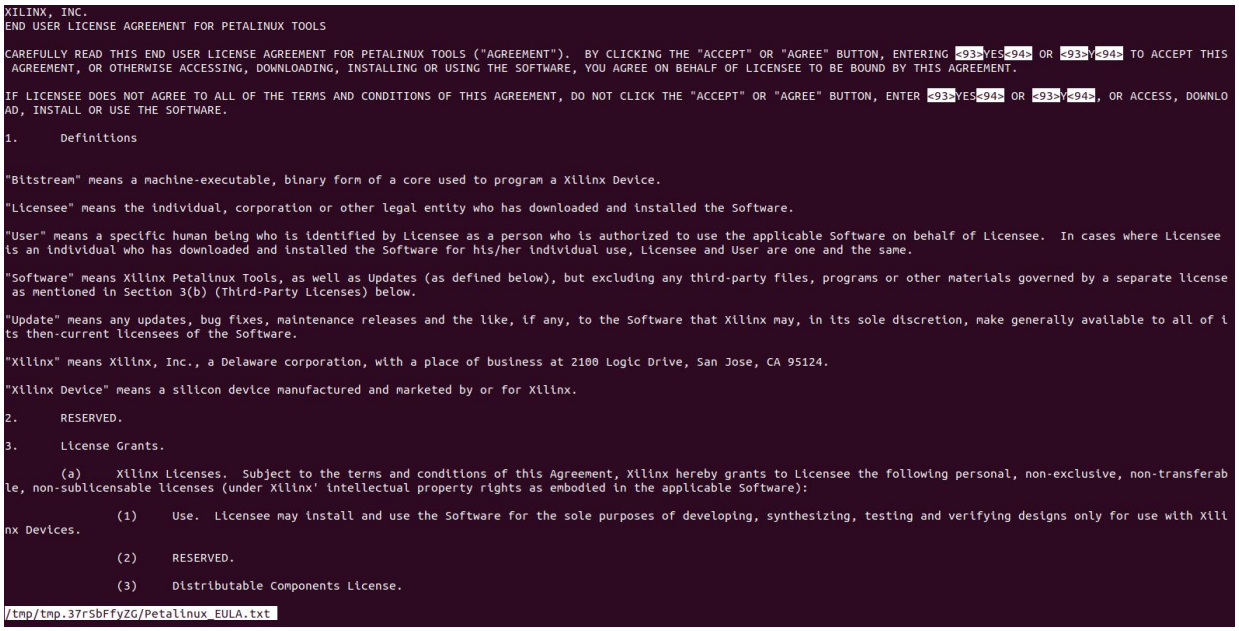
现在我们将 petalinux 安装在~/petalinux/2021.2 目录下，在终端中输入如下命令：

./petalinux-v2021.2-final-installer.run -d ~/petalinux/2021.2

执行 Petalinux 的安装，如下图所示：



从显示的意思可以知道，让我们按下回车键显示软件许可协议，按下键盘上的回车键 Enter，显示协议内容如下：



这些内容如果你感兴趣可以看看，直接按下键盘上的 Q 键退出，回到之前的界面之后会出现一个选择项，询问我们是否接受 xilinx 最终用户协议。

这个是没得选，必须接受，否则无法进行下面的安装；输入 y 按回车接受。除了 xilinx 最终用户协议之外，还有个协议也需要大家接受，会依次显示在终端上，同理也是输入 Y 按下回车接受。

```
sdr@sdr:~$ ./petalinux-v2021.2-final-installer.run -d ~/petalinux/2021.2
INFO: Checking installation environment requirements...
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "UG1144  PetaLinux Tools Documentation Reference Guide" for its impact and solution
INFO: Checking installer checksum...
INFO: Extracting PetaLinux installer...

LICENSE AGREEMENTS

PetaLinux SDK contains software from a number of sources.  Please review
the following licenses and indicate your acceptance of each to continue.

You do not have to accept the licenses, however if you do not then you may
not use PetaLinux SDK.

Use PgUp/PgDn to navigate the license viewer, and press 'q' to close

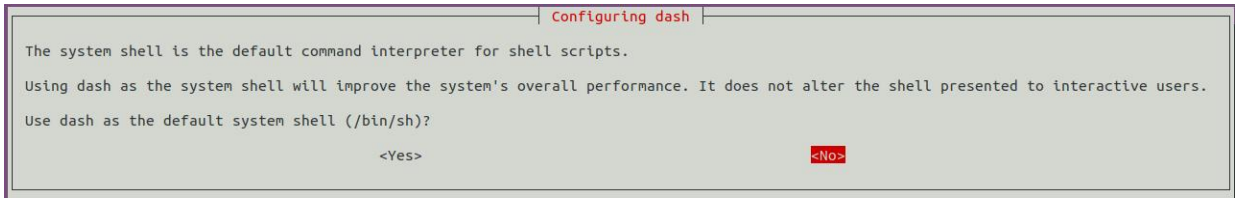
Press Enter to display the license agreements
Do you accept Xilinx End User License Agreement? [y/N] > y
Do you accept Third Party End User License Agreement? [y/N] > y
INFO: Installing PetaLinux...
```

接受所有协议之后 Petalinux 安装工具便会继续安装，直到安装完成。

Petalinux 工具需要主机系统的/bin/sh 是 bash，而 Ubuntu 默认的/bin/sh 是 dash，所以这里需要进行更改，运行 sudo dpkg-reconfigure dash 命令，如下：

sudo dpkg-reconfigure dash

执行结果如下图所示：



选择“No”按下回车即可。

1.4 设置 Petalinux 环境变量

在正式使用 petalinux 工具之前，需要先运行 petalinux 安装目录下的 settings.sh 脚本文件设置 petalinux 工作环境，

所需的运行环境进行配置，命令如下：

source ~/petalinux/2021.2/settings.sh

```
sdr@sdr:~$ source ~/petalinux/2021.2/settings.sh
PetaLinux environment set to '/home/sdr/petalinux/2021.2'
INFO: Checking free disk space
INFO: Checking installed tools
INFO: Checking installed development libraries
INFO: Checking network and other services
WARNING: No tftp server found - please refer to "UG1144 2021.2 PetaLinux Tools Documentation Reference Guide" for its impact and solution
sdr@sdr:~$
```

需要注意的是该命令只对当前终端有效，重新打开终端后需要重新执行这一步。

2、使用 Petalinux 定制 Linux 系统

可参考官网 petalinux 使用文档：PetaLinux Tools Documentation: Reference Guide (UG1144) (xilinx.com)

2.1 Petalinux 建立工程

1、为了方便工程的管理，我们在用户的家目录下新建一个 workspace/petalinux 目录，作为 petalinux 的工程目录。在终端中输入如下命令新建 workspace/petalinux 目录并切换到该目录：

mkdir -p ~/workspace/petalinux/

cd ~/workspace/petalinux/

```
sdr@sdr:~$ mkdir -p ~/workspace/petalinux/
sdr@sdr:~$ cd ~/workspace/petalinux/
sdr@sdr:~/workspace/petalinux$ ls
sdr@sdr:~/workspace/petalinux$
```

2、设置 petalinux 环境变量，运行下面命令，环境变量只在当前终端中有效，所以后面和 petalinux 相关的命令都要在当前终端中输入，新打开的终端需要重新运行：

source ~/petalinux/2021.2/settings.sh

3、创建 petalinux 工程，名称为“petalinux”，类型为“project”，使用 zynqMP 模板

petalinux-create -t project -n rfevm --template zynqMP

```
sdr@sdr:~/workspace/petalinux$ petalinux-create -t project -n rfevm --template zynqMP
INFO: Create project: rfevm
INFO: New project successfully created in /home/sdr/workspace/petalinux/rfevm
sdr@sdr:~/workspace/petalinux$ ls
rfevm
sdr@sdr:~/workspace/petalinux$
```

4、将 vivado 导出 xsa 文件拷贝到 ~/ workspace/petalinux/xsa 文件夹下。

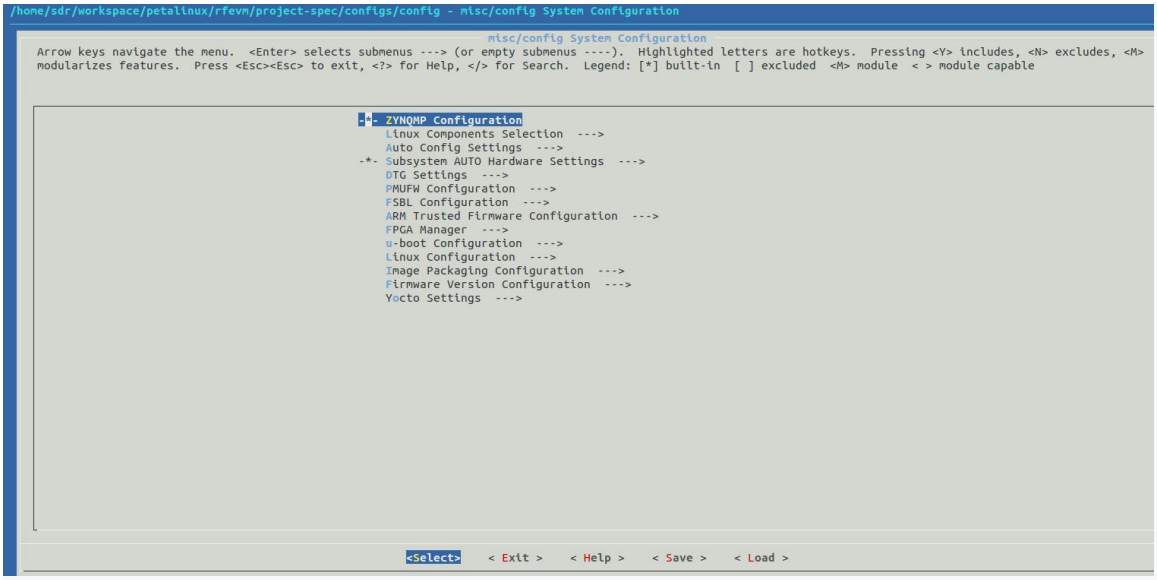
```
sdr@sdr:~/workspace/petalinux$ mkdir xsa
sdr@sdr:~/workspace/petalinux$ ls
rfevm  xsa
sdr@sdr:~/workspace/petalinux$
```

5、配置 petalinux 工程的硬件信息，硬件信息目录里只能有一个 xsa 文件

petalinux-config --get-hw-description ../xsa/

```
sdr@sdr:~/workspace/petalinux/rfevm$ petalinux-config --get-hw-description ../xsa/
[INFO] Sourcing buildtools
INFO: Getting hardware description...
INFO: Renaming system_wrapper.xsa to system.xsa
[INFO] Generating Kconfig for project
```

8、弹出工程配置项界面，如下图：



需要注意的是该窗口不可以使用鼠标操作，只能通过键盘操作，界面上方的英文就是简单的操作说明，操作方法如下：

通过键盘上的“↑”和“↓”键来选择要配置的菜单，按下“Enter”键进入子菜单。菜单中高亮的字母就是此菜单的热键，在键盘上按下此高亮字母对应的键可以快速选中对应的菜单。选中子菜单以后按下“Y”键就会将相应的配置选项写入配置文件中，菜单前面变为“<*>”。按下“N”键不编译相应的代码，按下“M”键就会将相应的代码编译为模块，菜单前面变为“<M>”。按两下“Esc”键退出，也就是返回到上一级，按下“?”键查看此菜单的帮助信息，按下“/”键打开搜索框，可以在搜索框输入要搜索的内容。

在配置界面下方会有五个按钮，这五个按钮的功能如下：

<Select>：选中按钮，和“Enter”键的功能相同，负责选中并进入某个菜单。

<Exit>：退出按钮，和按两下“Esc”键功能相同，退出当前菜单，返回到上一级。

<Help>：帮助按钮，查看选中菜单的帮助信息。

<Save>：保存按钮，保存修改后的配置文件。

<Load>：加载按钮，加载指定的配置文件。

本实验我们无需更改该窗口的配置信息。

然后通过按键选择“Exit”。然后等待配置完成。

```
sdr@sdr:~/workspace/petalinux/rfevm$ petalinux-config --get-hw-description ../xsa/
[INFO] Sourcing buildtools
INFO: Getting hardware description...
INFO: Renaming system_wrapper.xsa to system.xsa
[INFO] Generating Kconfig for project
[INFO] Menuconfig project
configuration written to /home/sdr/workspace/petalinux/rfevm/project-spec/configs/config

*** End of the configuration.
*** Execute 'make' to start the build or try 'make help'.

[INFO] Extracting yocto SDK to components/yocto. This may take time!
[INFO] Sourcing build environment
[INFO] Generating kconfig for Rootfs
[INFO] Silentconfig rootfs
[INFO] Generating plnxtool conf
[INFO] Adding user layers
[INFO] Generating workspace directory
sdr@sdr:~/workspace/petalinux/rfevm$
```

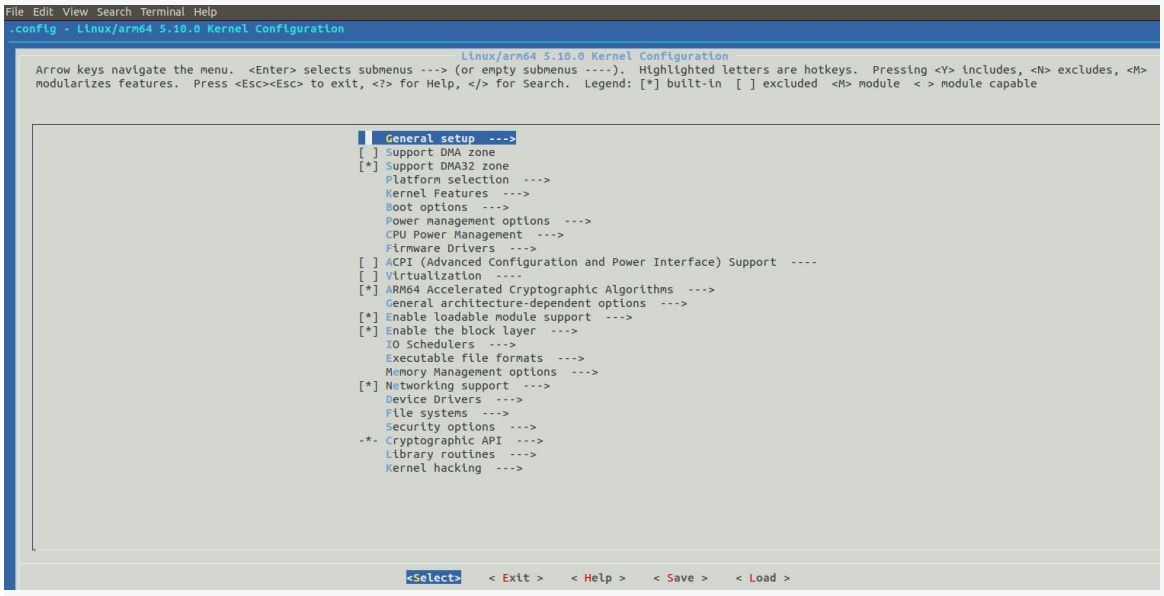
2.2 配置内核

1、现在我们开始定制 Linux 内核，在终端输入如下命令：

petalinux-config -c kernel

```
sdr@sdr:~/workspace/petalinux/rfevm$ petalinux-config -c kernel
[INFO] Sourcing buildtools
[INFO] Silentconfig project
[INFO] Sourcing build environment
[INFO] Generating kconfig for Rootfs
[INFO] Silentconfig rootfs
[INFO] Generating plnxtool conf
[INFO] Generating workspace directory
[INFO] Configuring: kernel
[INFO] bitbake virtual/kernel -c cleansstate
NOTE: Started PRServer with DBfile: /home/sdr/workspace/petalinux/rfevm/build/cache/prserv.sqlite3, IP: 127.0.0.1, PORT: 43297, PID: 8594
Loading cache: 100% |                                                                                                  | ETA:  --:--:--
Loaded 0 entries from dependency cache.
Parsing recipes:  19% |#########################                                                                       | ETA:  0:00:54
```

等一段时间后会在终端中创建一个名为“linux-xlnx Configuration”的标签页，也就是 Linux 内核的配置界面。



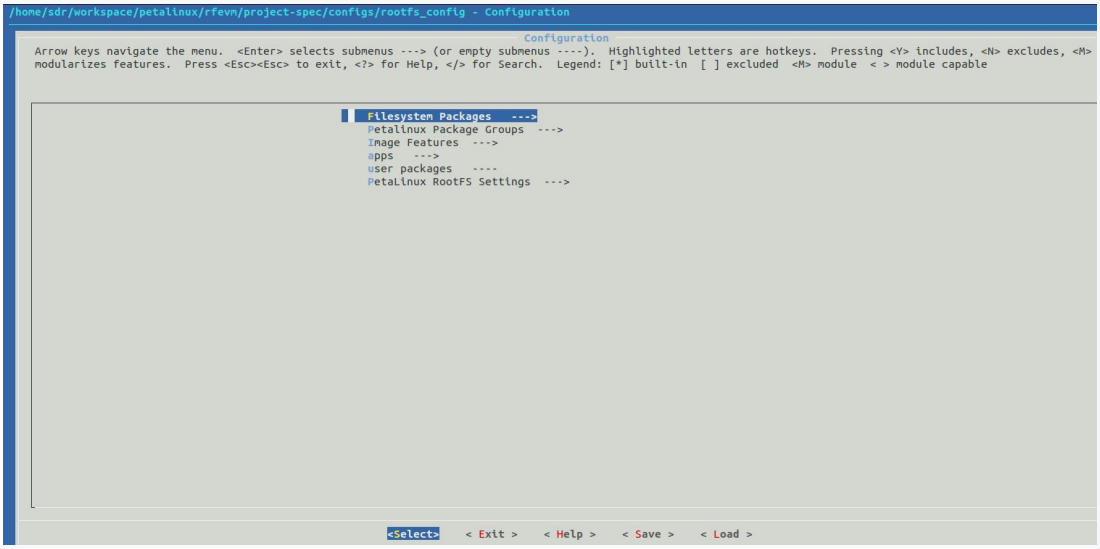
这里使用的内核 Xilinx 官方已经做好了基础配置，如无特定需求，无需更改。另外关于 Linux 内核的配置要根据硬件功能进行配置，此处就不多做介绍了。这里采用 Xilinx 官方的默认配置即可，保存配置并退出。

2.3 配置根文件系统

1、在终端输入下面的命令可配置根文件系统，如果不需要配置可不执行该命令。

petalinux-config -c rootfs

会出现一个配置界面



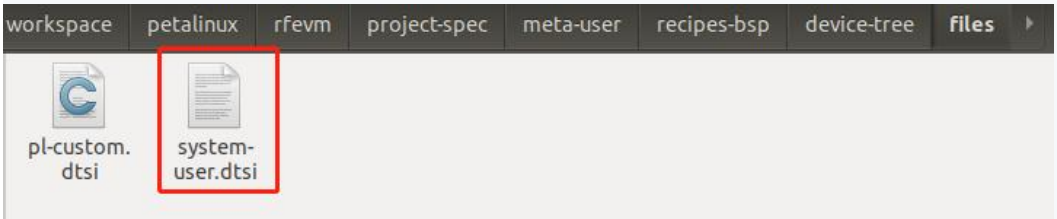
默认配置可满足一般使用，也可以根据需求来定制根文件系统，本实验保持默认配置。需要说明的是“PetaLinux RootFS Settings”可以用来设置 root 用户的密码，默认为“root”。后面登录的时候会用到。

保存配置并退出。

2.4 配置设备树

设备树用于保存 Linux 系统中的各种设备信息，内核在启动过程当中会去解析设备树文件，获取设备所需的配置信息完成设备的初始化工作。我们可以打开这个文件进行编辑，将外设信息添加到系统当中。

1、将工程根目录下 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 文件打开：



把 USB、硬盘、SD 卡和以太网设备树节点添加到 system-user.dtsi 设备树当中，system-user.dtsi 文件内容如下：

```
/include/ "system-conf.dtsi"
#include <dt-bindings/phy/phy.h>
#include <dt-bindings/gpio/gpio.h>
/ {
};
&psgtr {
	lane0: lane0 {
		#phy-cells = <4>;
	};
	lane1: lane1 {
		#phy-cells = <4>;
	};
	lane2: lane2 {
		#phy-cells = <4>;
	};
	lane3: lane3 {
		#phy-cells = <4>;
	};
};

&dwc3_0 {
	status = "okay";
	dr_mode = "host";
	phy-names = "usb3-phy";
	phys = <&lane2 PHY_TYPE_USB3 0 3 26000000>;
};

&dwc3_1 {
	status = "okay";
	dr_mode = "host";
	phy-names = "usb3-phy";
	phys = <&lane3 PHY_TYPE_USB3 1 3 26000000>;
};
```

```
&pcie {
        status = "okay";
        phys = <&lane0 PHY_TYPE_PCIE 0 0 100000000>, <&lane1 PHY_TYPE_PCIE 1 0
100000000>;
};

&sdhci0 {
        status = "okay";
        disable-wp;
        no-1-8-v;
};

&sdhci1 {
        status = "okay";
        disable-wp;
        no-1-8-v;
};

&gem3 {
        local-mac-address = [00 0a 35 00 11 55];
        phy-reset-gpio = <&gpio 44 GPIO_ACTIVE_LOW>;
        phy-reset-active-low;
};
```

2、输入命令，开始编译

petalinux-build



注：该步骤必须保证 ubuntu 系统可以连接网络， petalinux 需要从 github 上下载一些源代码。

这个编译过程还需要一些时间：

```
sdr@sdr:~/workspace/petalinux/rfevm$ petalinux-build
[INFO] Sourcing buildtools
[INFO] Building project
[INFO] Sourcing build environment
[INFO] Generating workspace directory
INFO: bitbake petalinux-image-minimal
NOTE: Started PRServer with DBfile: /home/sdr/workspace/petalinux/rfevm/build/cache/prserv.sqlite3, IP: 127.0.0.1, PORT: 45039, PID: 22599
Loading cache: 100% |################################################################################################################################################| Time: 0:00:00
Loaded 5116 entries from dependency cache.
Parsing recipes: 100% |##############################################################################################################################################| Time: 0:00:01
Parsing of 3476 .bb files complete (3459 cached, 17 parsed). 5133 targets, 226 skipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |###########################################################################################################################################| Time: 0:00:03
Checking sstate mirror object availability: 100% |###################################################################################################################| Time: 0:00:37
Sstate summary: Wanted 1221 Found 997 Missed 224 Current 86 (81% match, 82% complete)
NOTE: Executing Tasks
Currently 12 running tasks (0 of 0) 100% |                                                                                                                                                  |
0: autoconf-archive-2019.01.06-r0 do_populate_sysroot_setscene (pid 34173)  74% |#####################################################                    | 2.04M/s
1: libsm-1_1.2.3-r0 do_populate_sysroot_setscene (pid 34194)   0% |                                                                                                                                              |
2: libxcrypt-4.4.17-r0 do_packagedata_setscene (pid 34207)   0% |                                                                                                                                               |
3: flex-2.6.4-r0 do_packagedata_setscene (pid 34224)   0% |                                                                                                                                                  |
4: cracklib-2.9.5-r0 do_packagedata_setscene (pid 34237)   0% |                                                                                                                                              |
5: libx11-1_1.6.12-r0 do_populate_sysroot_setscene (pid 34253)  12% |##########                                                                                                                                  | 500K/s
6: initscripts-1.0-r155 do_deploy_source_date_epoch_setscene (pid 34269)   0% |                                                                                                                                             |
7: libpthread-stubs-0.4-r0 do_packagedata_setscene (pid 34275)   0% |                                                                                                                                               |
8: openssl-1.1.1k-r0 do_deploy_source_date_epoch_setscene (pid 34283)   0% |                                                                                                                                               |
9: expat-2.2.9-r0 do_deploy_source_date_epoch_setscene (pid 34302)   0% |                                                                                                                                                  |
10: update-rc.d-0.8-r0 do_deploy_source_date_epoch_setscene (pid 34313)   0% |                                                                                                                                               |
11: cmake-native-3.18.2-r0 do_populate_sysroot_setscene - 0s (pid 34334)
```

3、编译完成

```
sdr@sdr:~/workspace/petalinux/rfevm$ petalinux-build
[INFO] Sourcing buildtools
[INFO] Building project
[INFO] Sourcing build environment
[INFO] Generating workspace directory
INFO: bitbake petalinux-image-minimal
NOTE: Started PRServer with DBfile: /home/sdr/workspace/petalinux/rfevm/build/cache/prserv.sqlite3, IP: 127.0.0.1, PORT: 45039, PID: 22599
Loading cache: 100% |################################################################################################################################################| Time: 0:00:00
Loaded 5116 entries from dependency cache.
Parsing recipes: 100% |##############################################################################################################################################| Time: 0:00:01
Parsing of 3476 .bb files complete (3459 cached, 17 parsed). 5133 targets, 226 skipped, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies
Initialising tasks: 100% |###########################################################################################################################################| Time: 0:00:03
Checking sstate mirror object availability: 100% |###################################################################################################################| Time: 0:00:37
Sstate summary: Wanted 1221 Found 997 Missed 224 Current 86 (81% match, 82% complete)
NOTE: Executing Tasks
NOTE: Tasks Summary: Attempted 3986 tasks of which 2964 didn't need to be rerun and all succeeded.
INFO: Failed to copy built images to tftp dir: /tftpboot
[INFO] Successfully built project
sdr@sdr:~/workspace/petalinux/rfevm$
```

2.5 生成启动镜像

1、进入文件目录

cd ~/workspace/petalinux/rfevm/images/linux

2、打包文件，“--u-boot” 参数为要打包合成的内容， 会把 u-boot 依赖项全部合成:

--fsbl 指定生成的 fsbl 可执行文件

--fpga 指定由 hdf 文件中解压出的 bitstream 文件,如果你的 hdf 文件中未包含 bitstream，则不要加这个参数

--u-boot 指定生成的 u-boot 可执行文件

--force 为强制合成，覆盖以前合成的 BOOT.BIN

petalinux-package --boot --fsbl zynqmp_fsbl.elf --fpga system.bit --u-boot u-boot.elf --force

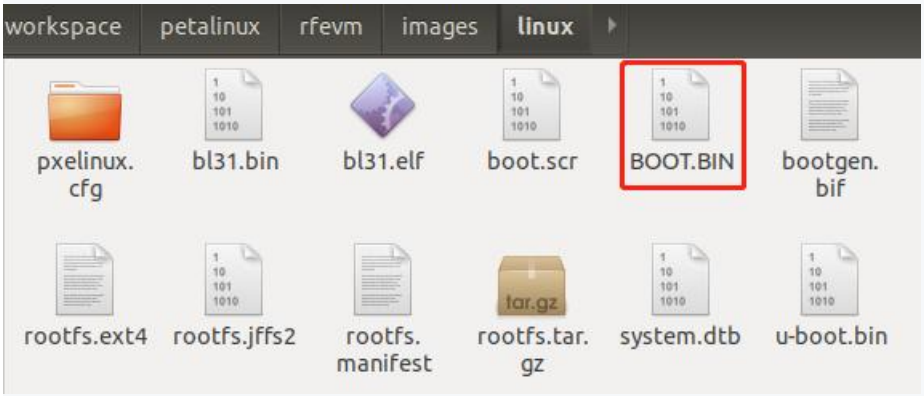
```
[INFO] Sourcing buildtools
INFO: Getting system flash information...
INFO: File in BOOT BIN: "/home/sdr/workspace/petalinux/rfevm/images/linux/zynqmp_fsbl.elf"
INFO: File in BOOT BIN: "/home/sdr/workspace/petalinux/rfevm/images/linux/pmufw.elf"
INFO: File in BOOT BIN: "/home/sdr/workspace/petalinux/rfevm/images/linux/bl31.elf"
INFO: File in BOOT BIN: "/home/sdr/workspace/petalinux/rfevm/images/linux/system.dtb"
INFO: File in BOOT BIN: "/home/sdr/workspace/petalinux/rfevm/images/linux/u-boot.elf"
INFO: Generating zynqmp binary package BOOT.BIN...


****** Xilinx Bootgen v2021.2
  **** Build date : Sep 30 2021-06:08:18
    ** Copyright 1986-2021 Xilinx, Inc. All Rights Reserved.


[INFO]   : Bootimage generated successfully

INFO: Binary is ready.
WARNING: Unable to access the TFTPBOOT folder /tftpboot!!!
WARNING: Skip file copy to TFTPBOOT folder!!!
```

3、在当前目录下，可以看到新生成的 BOOT.bin 文件



2.4 启动系统

1、格式化 SD 卡为 FAT32 格式

2、将文件 BOOT.bin、boot.scr 和 image.ub 拷贝到 SD 卡



3、SD 卡插入板卡

4、板卡设置为 SD 卡启动模式

5、打开串口终端并将板卡上电

6、用户名 root，密码 root 登录，进入系统，因为根文件系统为默认的 ram 类型，文件系统不能保存文件和配置。

```
Starting Dropbear SSH server: Waiting for kernel randomness to be initialised...
haveged: haveged: ver: 1.9.13; arch: generic; vend: ; build: (gcc 10.2.0 CTV); collect: 128K

haveged: haveged: cpu: (VC); data: 16K (D); inst: 16K (D); idx: 11/40; sz: 15456/64452

haveged: haveged: tot tests(BA8): A:1/1 B:1/1 continuous tests(B):  last entropy estimate 7.99979

haveged: haveged: fills: 0, generated: 0

[   15.071997] random: crng init done
Generating 2048 bit rsa key, this may take a while...
[   15.075418] random: 7 urandom warning(s) missed due to ratelimiting
Public key portion is:
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDlmUR42SfwgAjLdPfWVx8L/Z1klVxxJfzzRW9N8EAbq9MHaa7HR6/2oBDvw/gJ5H2s
B6hPmYp45/i78tkCICRqLUfWzpR+iUQq0CSDKizoGXLZ9xyO09/crgMCEmQnGed2PG/iNn6+GAtl4MHnoV7q+R8zt4DpkPlJ79Wqh8Bu
K+3jl6UXZpJE/LGLK/j1xhXTxYExE0D+zLCoTAk1v0G5qsygI0VqweNn/RS4LX3IqIfrL54UkIWWMi1uX1t4+S3JJ/Bsy5X9rypdvcWW
I/AQE0edJdmR9axYg7WzPHtCGUHU9n9Sn4Crt5UjhqnOlRZ root@rfevm
Fingerprint: sha1!! 18:6a:08:ae:7f:f2:72:a6:fb:bd:d0:42:da:0d:72:f0:e1:77:1f:41
dropbear.
Starting internet superserver: inetd.
Starting syslogd/klogd: done
Starting tcf-agent: OK

root@rfevm:~#
root@rfevm:~#
```

2.5 保存一个工程为模板工程

创建 bsp：petalinux-package --bsp -p rfevm --output V3Best-RFEVM-2021.2.bsp

```
sdr@sdr:~/workspace/petalinux$ petalinux-package --bsp -p rfevm --output V3Best-RFEVM-2021.2.bsp
/home/sdr/workspace/petalinux/V3Best-RFEVM-2021.2.bsp
INFO: Target BSP "/home/sdr/workspace/petalinux/V3Best-RFEVM-2021.2.bsp" will contain the following projects
INFO: PetaLinux project: /home/sdr/workspace/petalinux/rfevm
INFO: Sourcing build environment
INFO:   Copying /home/sdr/workspace/petalinux/rfevm/config.project
INFO:   Copying /home/sdr/workspace/petalinux/rfevm/.petalinux
INFO:   Copying /home/sdr/workspace/petalinux/rfevm/.gitignore
INFO:   Copying /home/sdr/workspace/petalinux/rfevm/project-spec
INFO:   Copying /home/sdr/workspace/petalinux/rfevm/components
INFO: Creating BSP
INFO: Generating package V3Best-RFEVM-2021.2.bsp...
INFO: BSP is ready
sdr@sdr:~/workspace/petalinux$
sdr@sdr:~/workspace/petalinux$ ls
rfevm  V3Best-RFEVM-2021.2.bsp  xsa
sdr@sdr:~/workspace/petalinux$
```

-p：工程目录

--output：输出的 bsp 文件名

3、配置系统默认 IP

为方便程序的开发和调试，我们可以设置系统启动的默认 IP。

3.1 创建 meta-user layer

1、执行命令：

mkdir -p project-spec/meta-user/recipes-core/init-ifupdown

cd project-spec/meta-user/recipes-core/init-ifupdown

```
sdr@sdr:~/workspace/petalinux/rfevm$
sdr@sdr:~/workspace/petalinux/rfevm$ mkdir -p project-spec/meta-user/recipes-core/init-ifupdown
sdr@sdr:~/workspace/petalinux/rfevm$ cd project-spec/meta-user/recipes-core/init-ifupdown
sdr@sdr:~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown$ ls
sdr@sdr:~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown$
```

2、新建项目：

vim init-ifupdown_%.bbappend

并填入以下内容：

```
SRC_URI += " \
            file://interfaces \
            "

FILESEXTRAPATHS_prepend := "${THISDIR}/files:"


# Overwrite interface file with interface file in rootfs
do_install_append() {
    install -m 0644 ${WORKDIR}/interfaces ${D}${sysconfdir}/network/interfaces
}
```

```
sdr@sdr: ~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown
File  Edit  View  Search  Terminal  Help
SRC_URI += " \
            file://interfaces \
            "

FILESEXTRAPATHS_prepend := "${THISDIR}/files:"

# Overwrite interface file with interface file in rootfs
do_install_append() {
    install -m 0644 ${WORKDIR}/interfaces ${D}${sysconfdir}/network/interfaces
}
```

保存退出即可。

3、最后创建 files 文件夹，并进入该目录下创建文件 interfaces：

mkdir files

cd files

vim interfaces

```
sdr@sdr:~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown$
sdr@sdr:~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown$ mkdir files
sdr@sdr:~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown$ ls
files  init-ifupdown_%.bbappend
sdr@sdr:~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown$ cd files/
sdr@sdr:~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown/files$ ls
sdr@sdr:~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown/files$
sdr@sdr:~/workspace/petalinux/rfevm/project-spec/meta-user/recipes-core/init-ifupdown/files$ vim interfaces
```

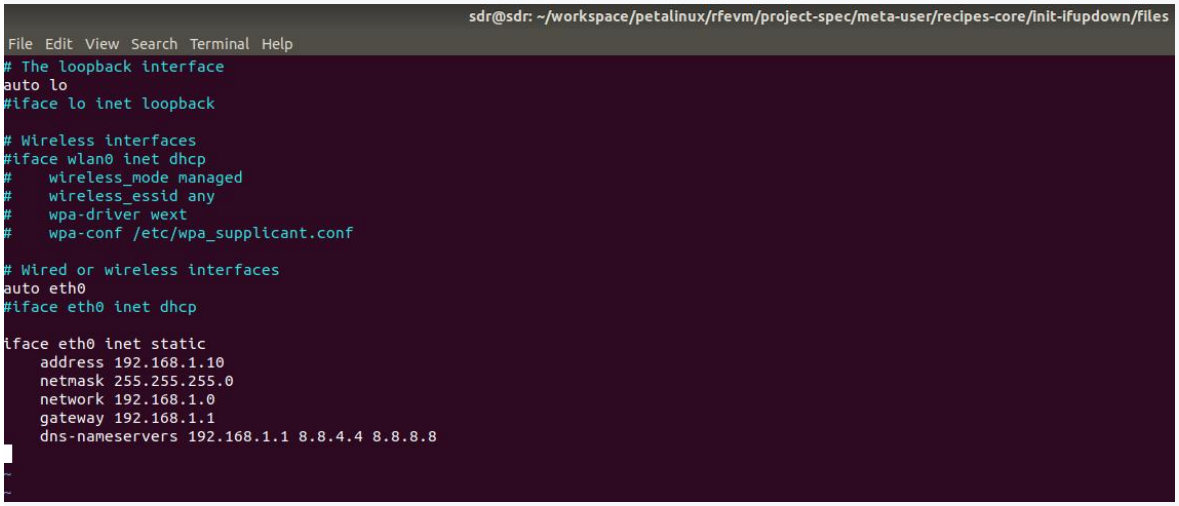
并填入以下内容：

```
# The loopback interface
auto lo
#iface lo inet loopback
```

```
# Wireless interfaces
#iface wlan0 inet dhcp
#       wireless_mode managed
#       wireless_essid any
#       wpa-driver wext
#       wpa-conf /etc/wpa_supplicant.conf

# Wired or wireless interfaces
auto eth0
#iface eth0 inet dhcp

iface eth0 inet static
        address 192.168.1.10
        netmask 255.255.255.0
        network 192.168.1.0
        gateway 192.168.1.1
        dns-nameservers 192.168.1.1 8.8.4.4 8.8.8.8
```



保存退出即可。

3.2 编译工程

重新编译工程：

```
petalinux-build
```

最后重新生成镜像文件，拷贝到 SD 卡重新启动即可。

```
root@rfevm:/media/card0# ifconfig
eth0      Link encap:Ethernet  HWaddr 62:A8:DA:BC:8B:96
          inet addr:192.168.1.10  Bcast:0.0.0.0  Mask:255.255.255.0
          inet6 addr: fe80::60a8:daff:febc:8b96/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:116 errors:0 dropped:0 overruns:0 frame:0
          TX packets:83 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:7963 (7.7 KiB)  TX bytes:10076 (9.8 KiB)
          Interrupt:36

root@rfevm:/media/card0#
```

4、添加开机自启动脚本及用户文件

前面的教程中我们使用 petalinux 来创建 Linux，默认根文件系统为 ramdisk 类型，对该目录下的任何修改，在系统重启后又会丢失，本章讲述如果添加用户的应用及数据文件开机不丢失，并开机自启动用户的应用程序。

4.1 自启动脚本创建

1、控制终端中进入工程跟目录

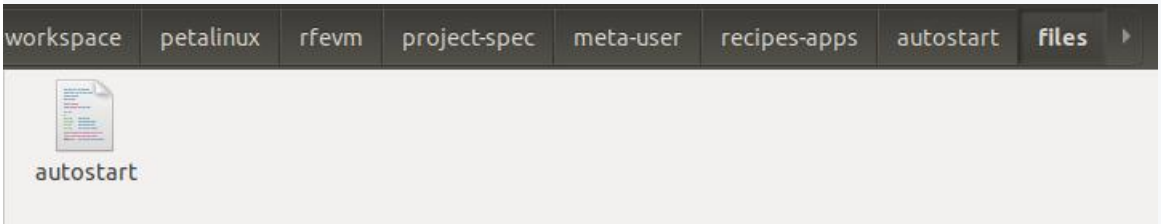
cd ~/workspace/petalinux/rfevm

2、输入如下命令，创建一个应用程序模板，创建一个名为“autostart”的应用程序

petalinux-create -t apps --template install --name autostart --enable

```
sdr@sdr:~/workspace/petalinux/rfevm$ ls
build  components  config.project  images  project-spec
sdr@sdr:~/workspace/petalinux/rfevm$ petalinux-create -t apps --template install --name autostart --enable
INFO: Create apps: autostart
INFO: New apps successfully created in /home/sdr/workspace/petalinux/rfevm/project-spec/meta-user/recipes-apps/autostart
INFO: Enabling created component...
INFO: Sourcing build environment
INFO: Silentconfig rootfs
INFO: autostart has been enabled
sdr@sdr:~/workspace/petalinux/rfevm$
```

3、进入模板目录



4、打开 files 文件夹下的 autostart 文件，这是一个脚本文件，增加打印字符串”check autostart.sh”，然后添加一段判断 SD 第一分区是否存在 autostart.sh 脚本， 如果存在就运行，用户可以根据实际需要迸行修改，如启动应用程序或其他初始化操作。

USERHOOK_SD0=/media/sd-mmcblk0p1/autostart.sh
USERHOOK_SD1=/media/sd-mmcblk1p1/autostart.sh
if [-f $USERHOOK_SD0]; then
sh $USERHOOK_SD0 &
fi
if [-f $USERHOOK_SD1]; then
sh $USERHOOK_SD1 &
fi

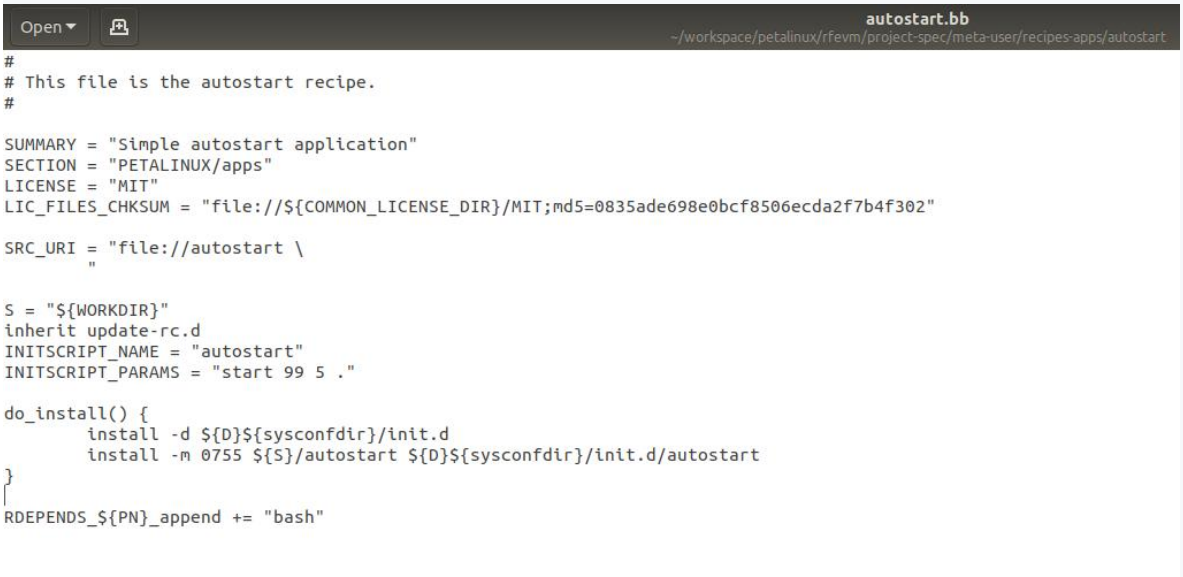




5、编辑文件 autostart.bb 内容如下，主要功能是复制 autostart 到根文件系统 init.d 目录，然后添加 0755 权限。RDEPENDS_${PN}_append += "bash"指这个要依赖 bash

#
This file is the autostart recipe.
#
SUMMARY = "Simple autostart application"
SECTION = "PETALINUX/apps"
LICENSE = "MIT"
LIC_FILES_CHKSUM =
"file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

```
SRC_URI = "file://autostart \
"
S = "${WORKDIR}"
inherit update-rc.d
INITSCRIPT_NAME = "autostart"
INITSCRIPT_PARAMS = "start 99 5 ."
do_install() {
install -d ${D}${sysconfdir}/init.d
install -m 0755 ${S}/autostart ${D}${sysconfdir}/init.d/autostart
}
RDEPENDS_${PN}_append += "bash"
```



4.2 编译工程

重新编译程序

petalinux-build

并重新生成启动镜像。

4.3 自启动测试

打包应用程序并启动，这里看到，添加的脚本已经启动，还可以在 sd 卡第一分区放一个 autostart.sh 的脚本， 脚本里面要干什么， 我们可以尽情发挥了。



在 autostart.sh 脚本中添加一行打印信息，开机启动后如下图所示：

```
#!/bin/sh

echo "hello world rfevm"
```

```
Public key portion is:
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQCZTypcrVfhMa0s4jG
NSLW6CYMTSY08lduv5O6b5Mwxn/UqRoKyBYjWK7aDeW+a4SDQoz1bKq
6jzaeFYAbyabKC+5k0ZaeqSv3ZCHRfM+hMTeUV2INBpZmJ0FejcNlcL
QAXglG4j6+OK3SbNSuVGTN0MWsLEKPX/xocMhZC3VAlR root@rfevm
Fingerprint: sha1!! 14:e1:10:1c:b7:79:35:54:e8:eb:76:87
dropbear.
Starting internet superserver: inetd.
Starting syslogd/klogd: done
Hello PetaLinux World
hello world rfevm
Starting tcf-agent: OK

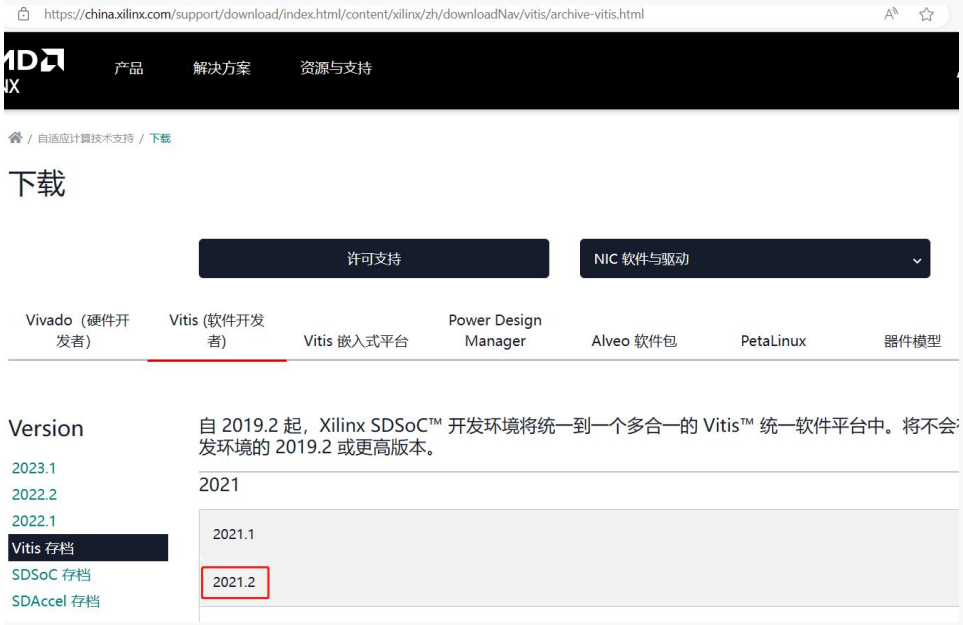
root@rfevm:~#
root@rfevm:~#
```

5、使用 Vitis 开发 Linux 应用

前面学习了在 Petalinux 搭建的 Linux 系统上基础外设的使用，本章我们以创建“Hello World”工程为例学习如何使用 Vitis 开发 Linux 应用以及如何让应用程序运行在 Petalinux 搭建的 Linux 系统上。

5.1 Vitis 安装

1、首先在官网进行下载安装包，下载的版本要和 petalinux 统一起来，因此也需要下载 2021.2 版本的。

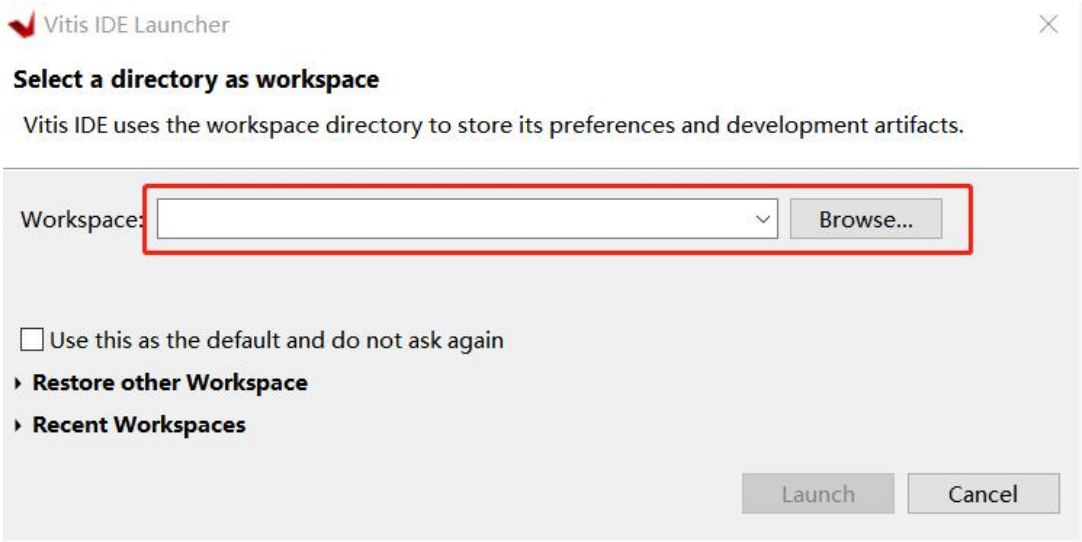


2、下载完成之后，直接在 window 下进行安装即可。

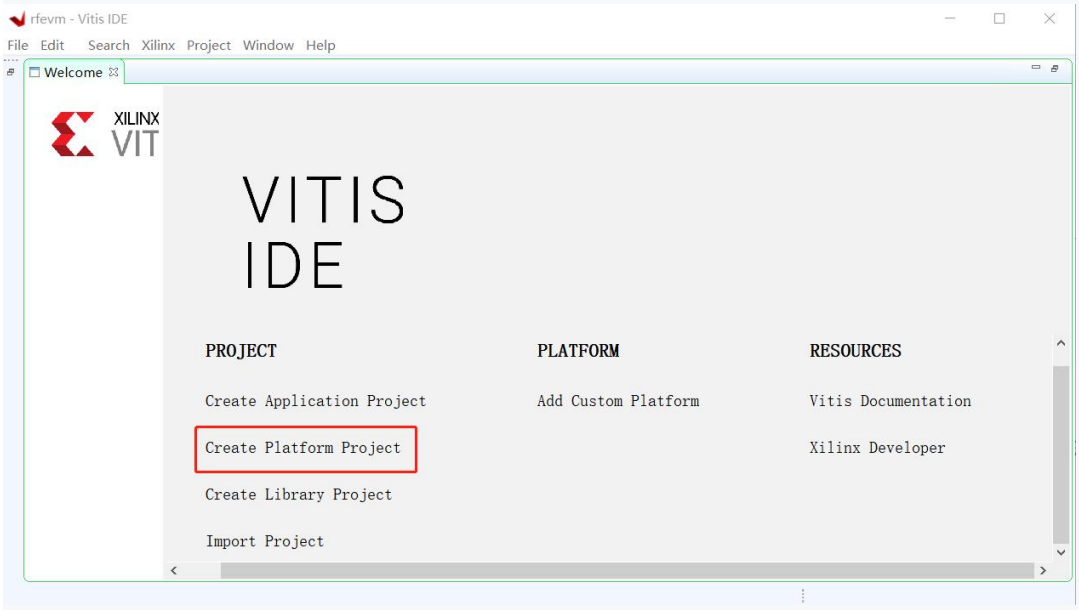
5.2 创建基于 Vitis 的 Linux 平台工程

1、vitis 这个开发工具，当你需要创建应用程序的时候，你需要创建一个基于 linux 的平台，这个平台依托于 vivado 工程，需要用到其生成的 xsa 文件。

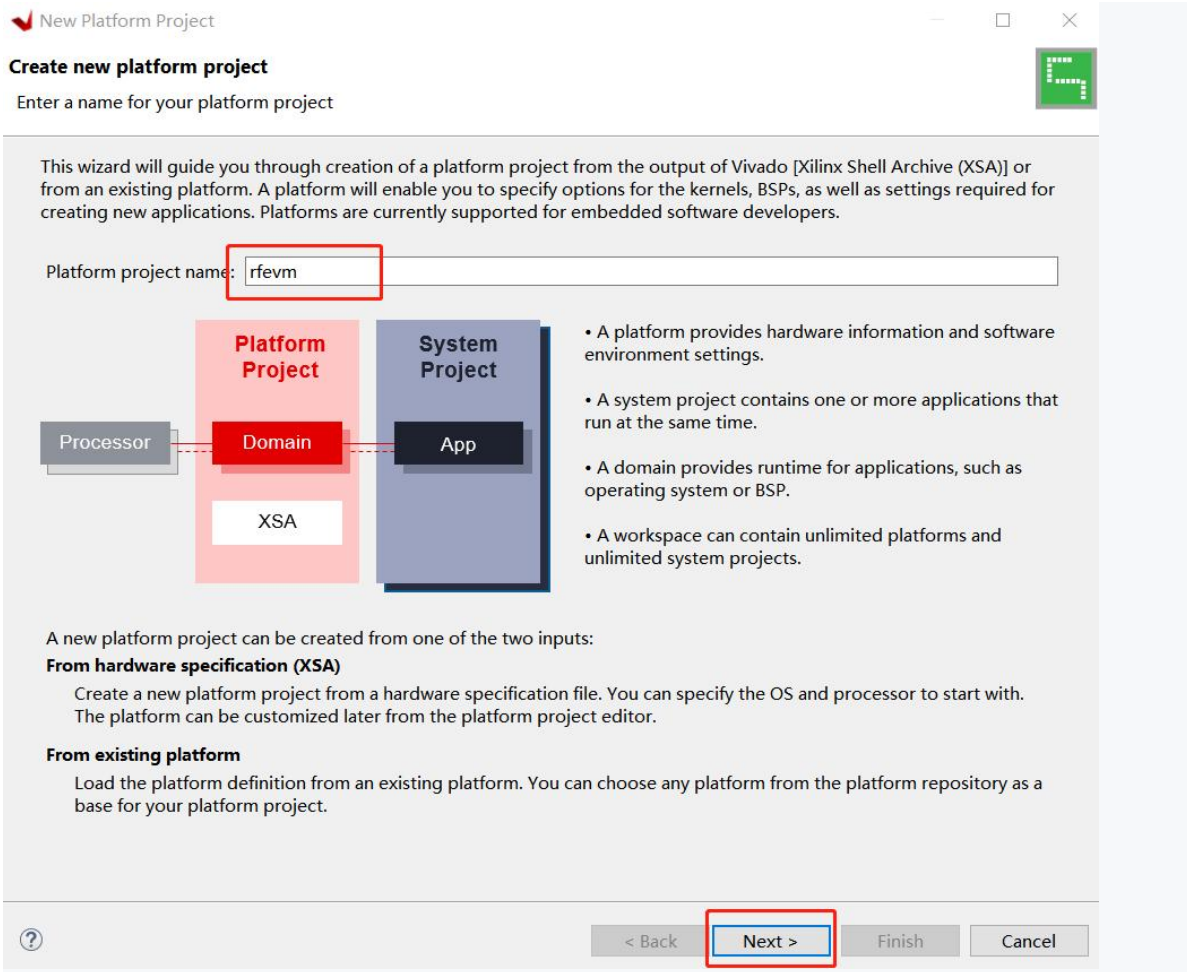
2、双击 Windows 系统桌面的 Xilinx Vitis 2021.2 图标打开 Vitis 工具，弹出下图所示的 Lancher 界面：



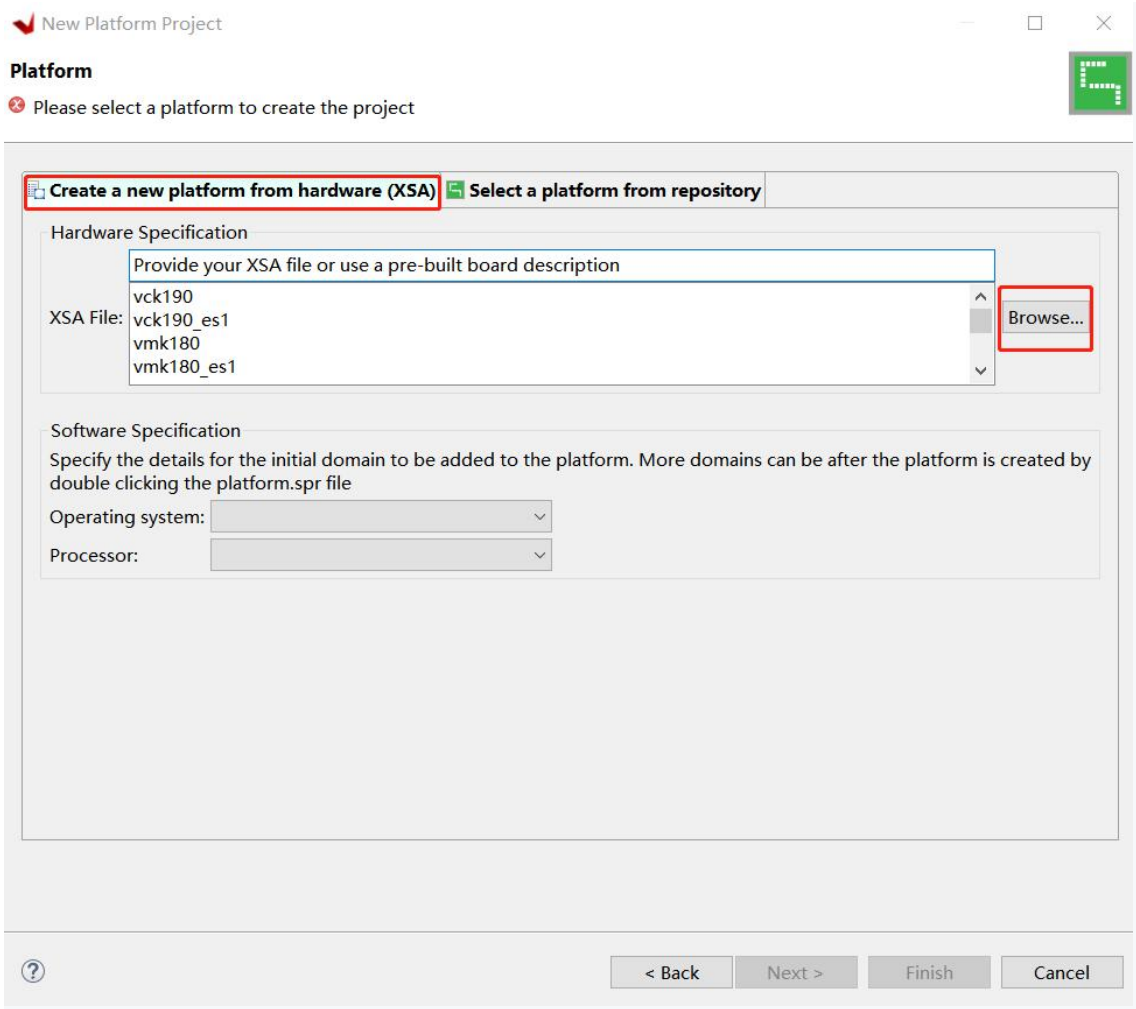
在 Workspace 创建工程路径，可以是任一文件夹，不要包括中文以及特殊字符，这里笔者选择的是 E:\Mpsoc\rfevm 文件夹，然后点击“Launch”按钮，进入如下图所示界面：



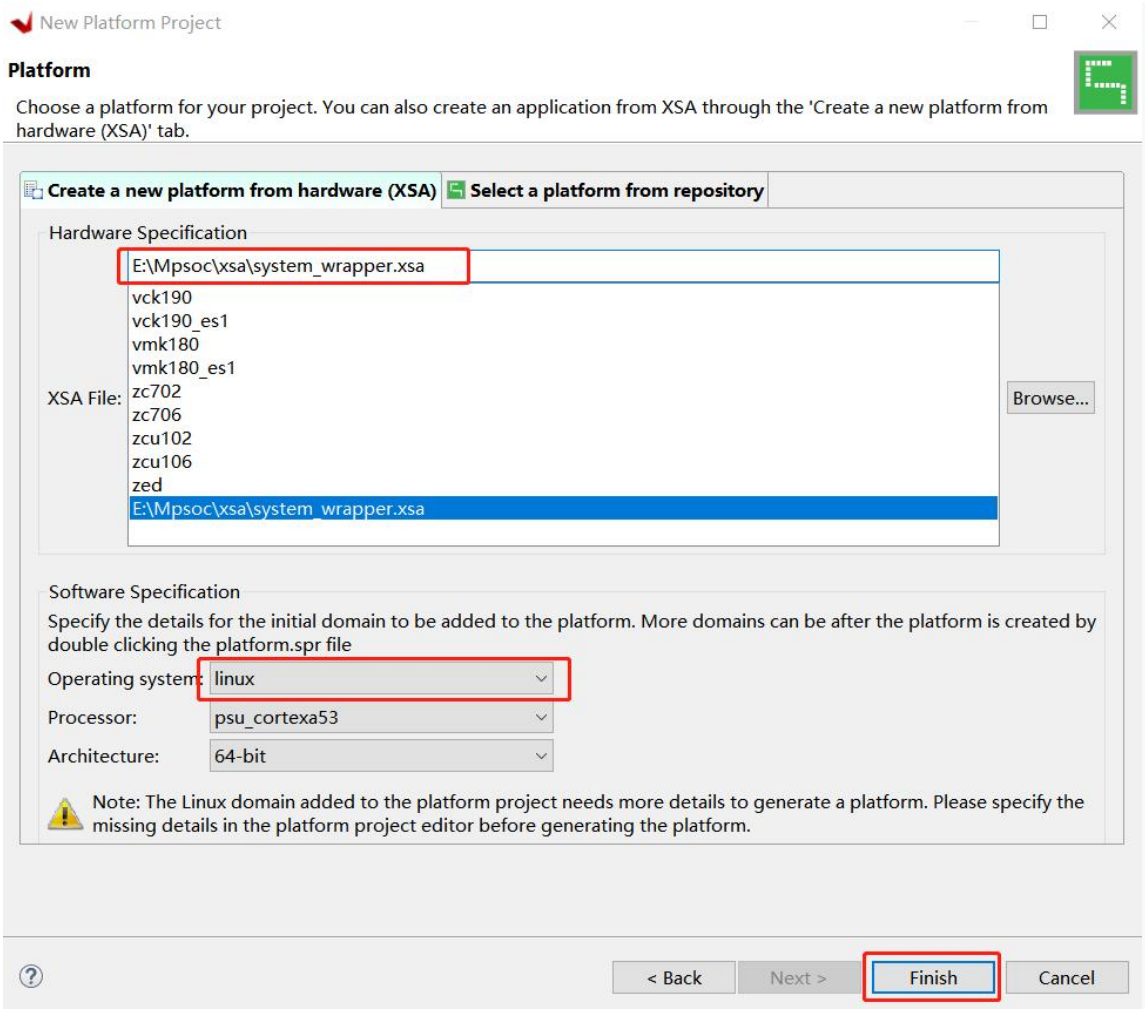
单击“Create Platform Project”,即创建平台工程，进入下图所示界面：



“Project name”栏输入平台工程名“rfevm”，其他保持默认，然后单击底部的“Next >”



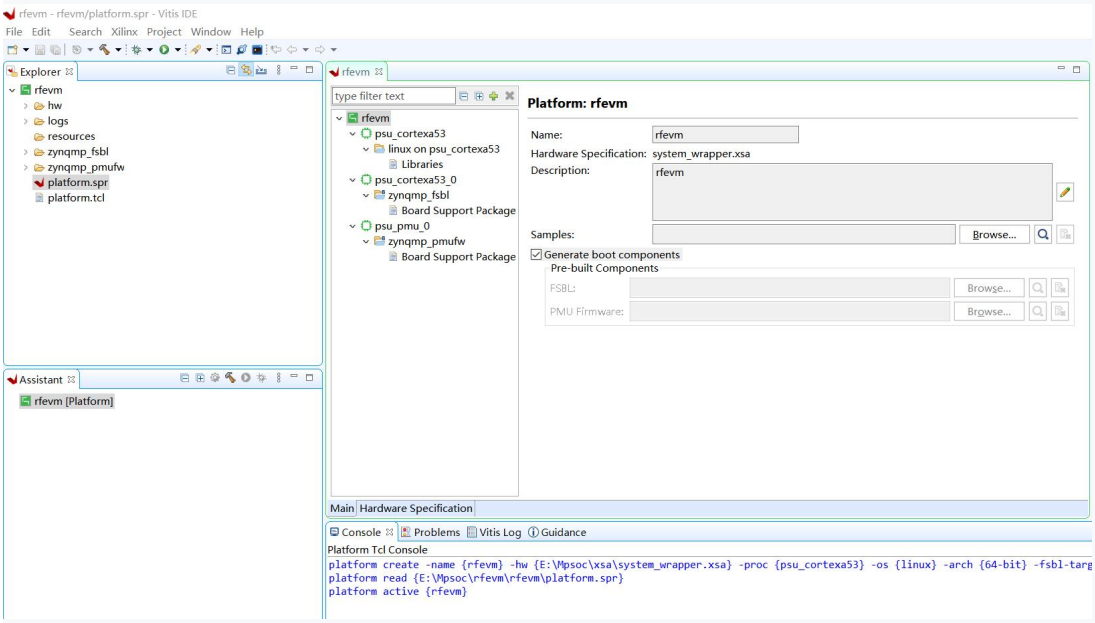
选择 “Create a new platform from hardware (XSA)”，然后点击 Browse 选择 xsa 文件：



“XSA file:”栏选择用于该平台工程的 xsa 文件，。

“Operating system”操作系统选择“linux”，其他使用默认配置。

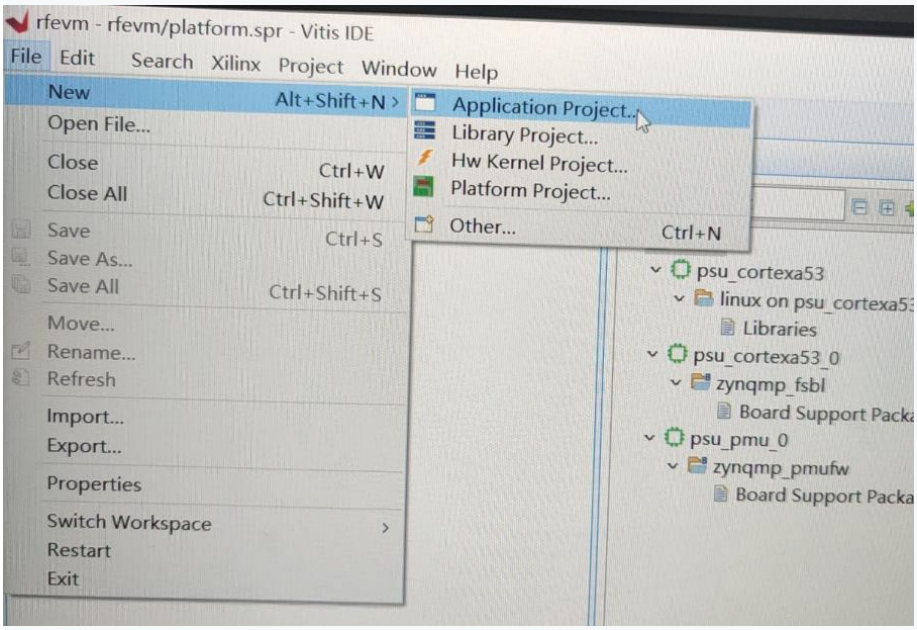
配置好后，点击底部的“Finish”按钮。完成之后就会出现如下界面：

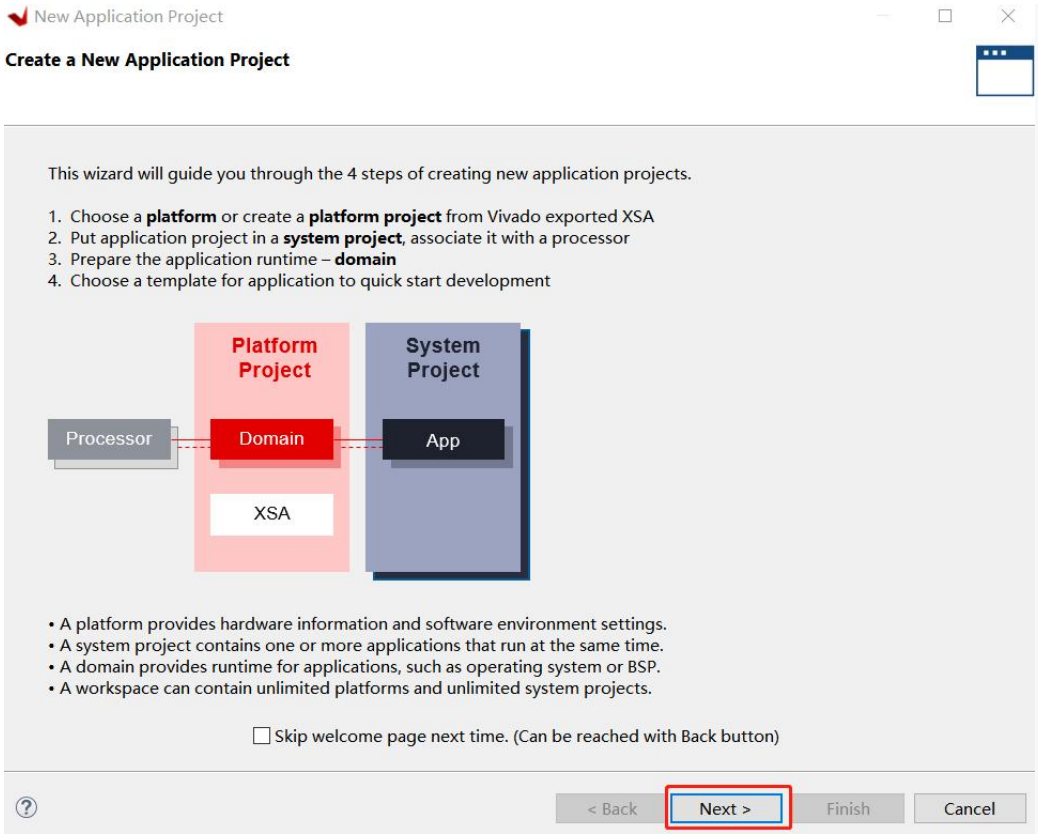


然后进行编译平台工程，编译完成之后，平台工程的搭建就完成了。现在可以创建 Linux 应用工程了。

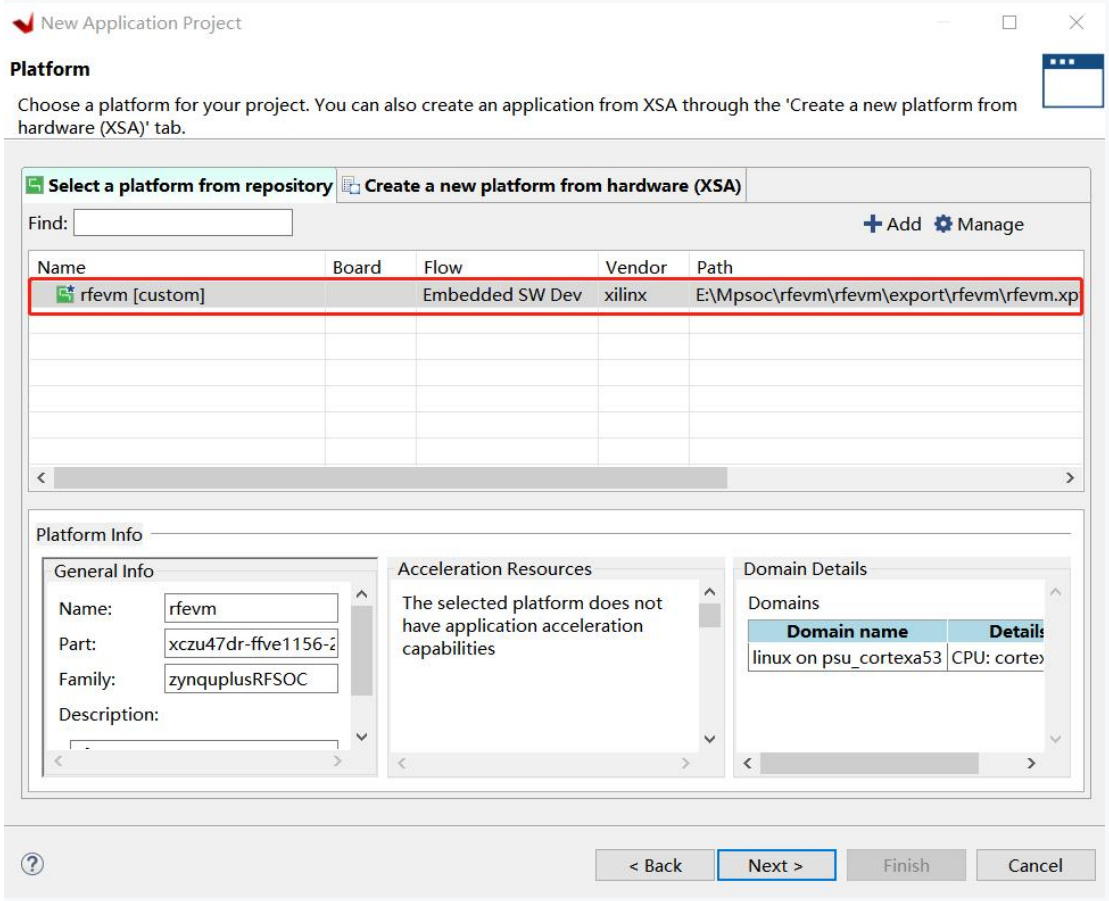
5.3 创建 Linux 应用工程

1、点击 Vitis 菜单栏的“File->New->Application Project…”，创建应用工程，如下图所示：

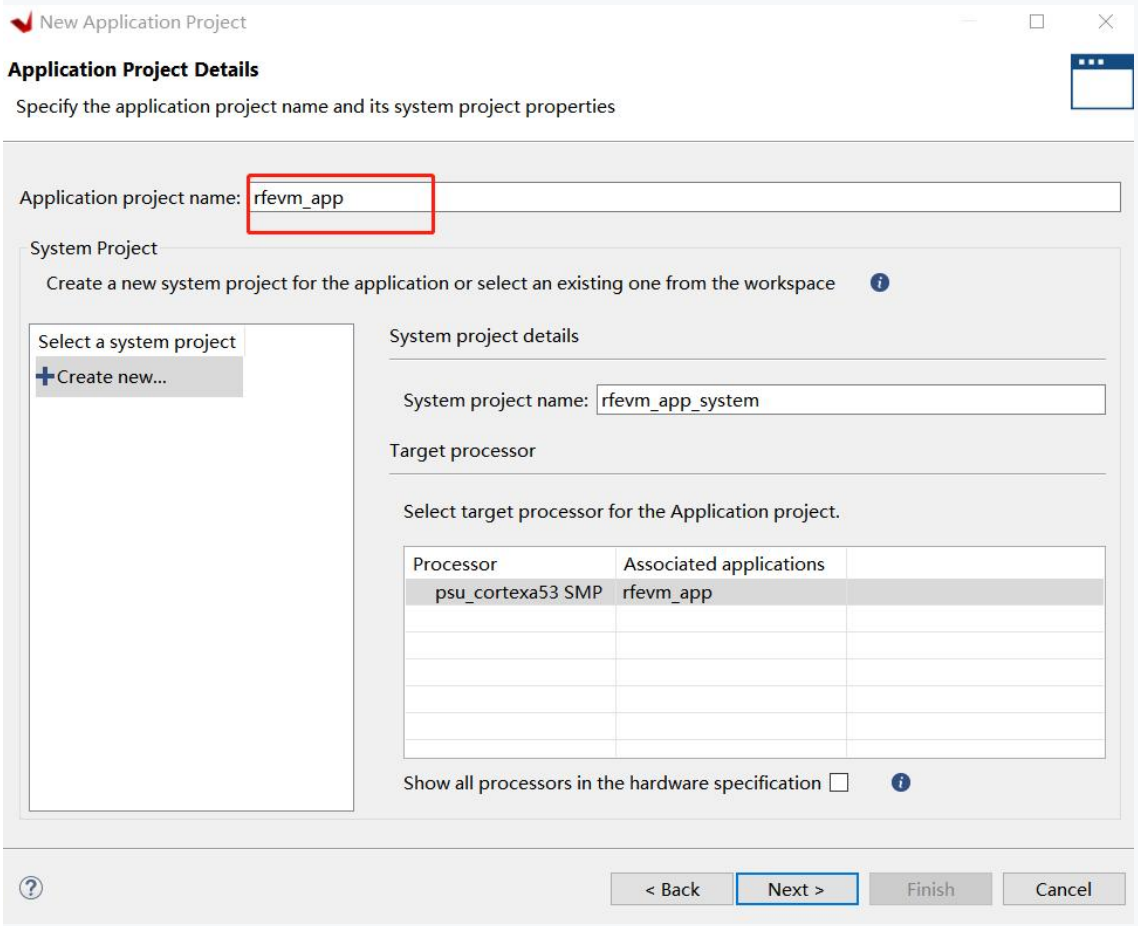



New Application Project
Create a New Application Project
This wizard will guide you through the 4 steps of creating new application projects.
1. Choose a platform or create a platform project from Vivado exported XSA
2. Put application project in a system project, associate it with a processor
3. Prepare the application runtime – domain
4. Choose a template for application to quick start development
Platform Project
System Project
Processor
Domain
App
XSA
• A platform provides hardware information and software environment settings.
• A system project contains one or more applications that run at the same time.
• A domain provides runtime for applications, such as operating system or BSP.
• A workspace can contain unlimited platforms and unlimited system projects.
Skip welcome page next time. (Can be reached with Back button)
< Back
Next >
Finish
Cancel

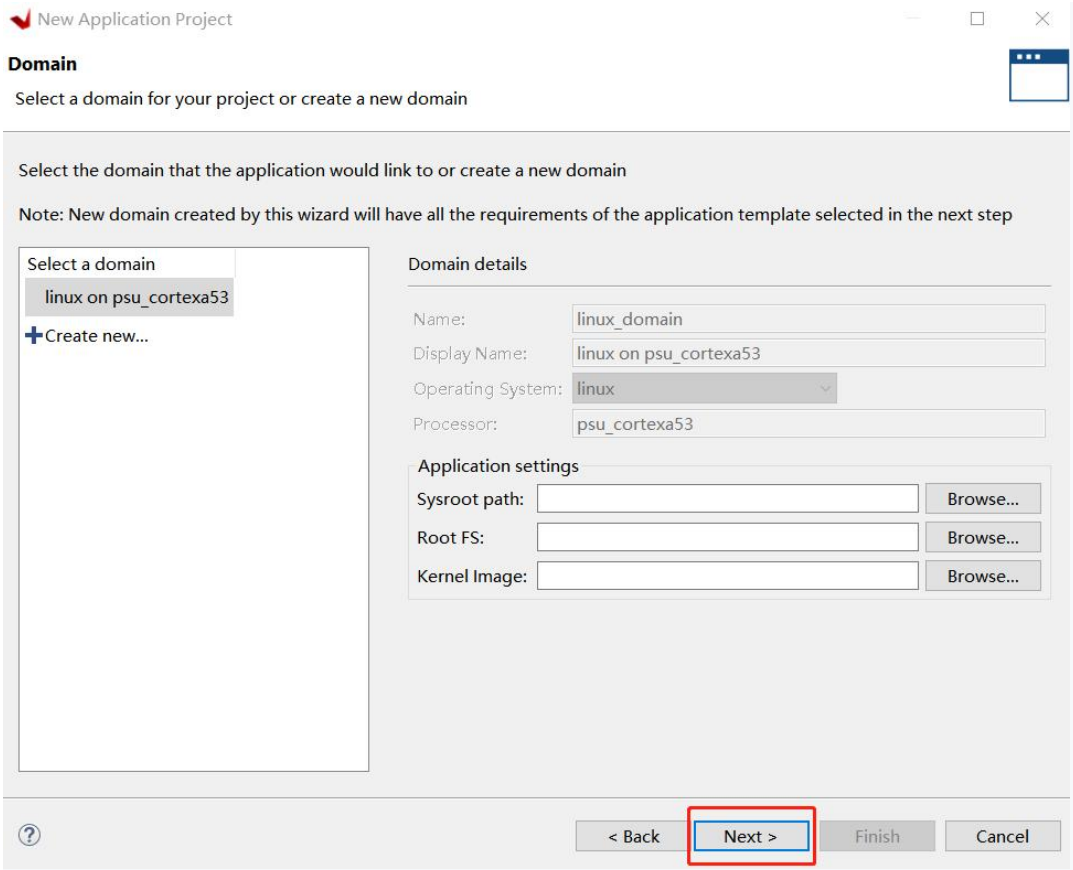
然后直接点击“Next”：


New Application Project
Platform
Choose a platform for your project. You can also create an application from XSA through the 'Create a new platform from hardware (XSA)' tab.
Select a platform from repository
Create a new platform from hardware (XSA)
Find:
Add
Manage
Name
Board
Flow
Vendor
Path
rfevm [custom]
Embedded SW Dev
xilinx
E:\Mpsoc\rfevm\rfevm\export\rfevm\rfevm.xp
Platform Info
General Info
Name: rfevm
Part: xczu47dr-ffve1156-2
Family: zynquplusRFSOC
Description:
Acceleration Resources
The selected platform does not have application acceleration capabilities
Domain Details
Domains
Domain name
Details
linux on psu_cortexa53
CPU: cortex
< Back
Next >
Finish
Cancel

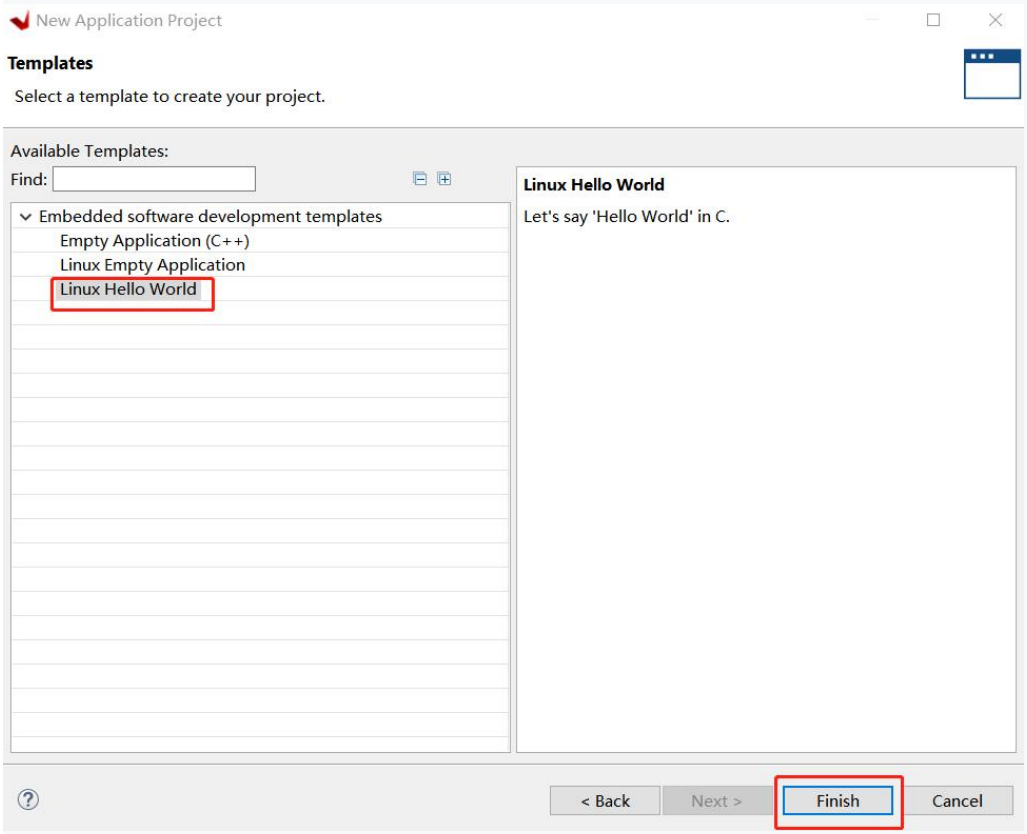
选择平台，然后点击“Next”：



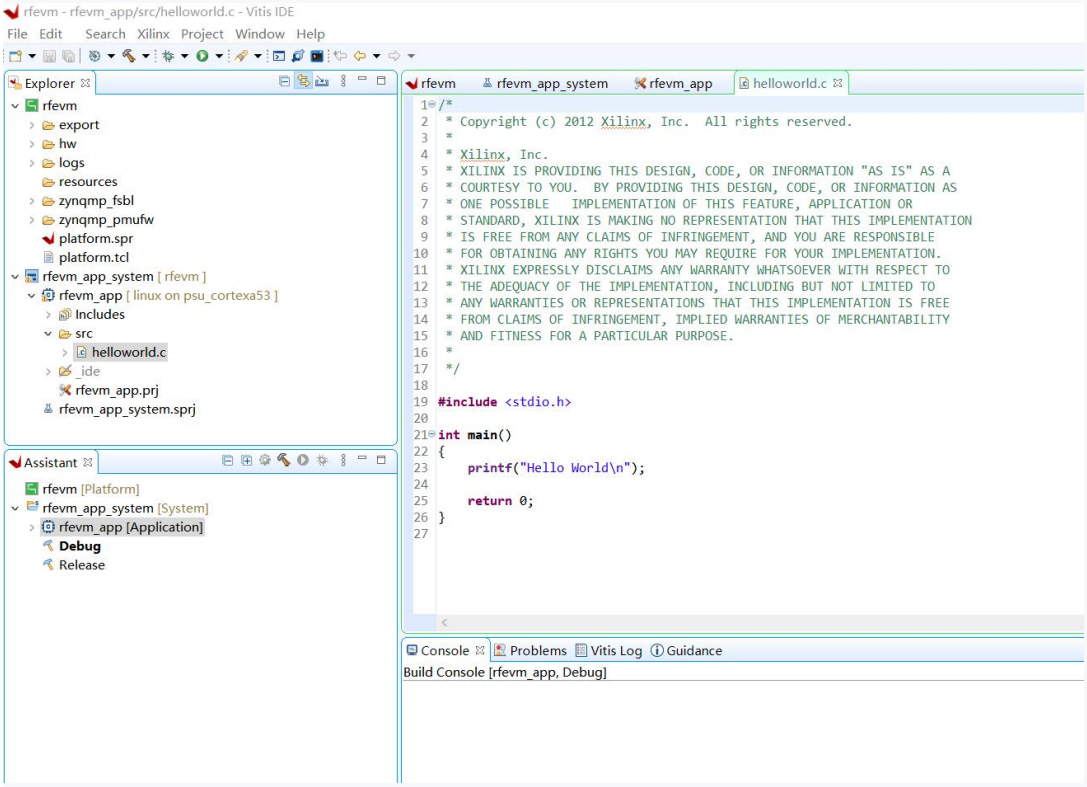
设置工程名为“rfevm_app”，然后直接“Next”：


New Application Project
Domain
Select a domain for your project or create a new domain
Select the domain that the application would link to or create a new domain
Note: New domain created by this wizard will have all the requirements of the application template selected in the next step
Select a domain
linux on psu_cortexa53
Create new...
Domain details
Name: linux_domain
Display Name: linux on psu_cortexa53
Operating System: linux
Processor: psu_cortexa53
Application settings
Sysroot path:
Root FS:
Kernel Image:
Browse...
< Back
Next >
Finish
Cancel

然后直接点击“Next”：


New Application Project
Templates
Select a template to create your project.
Available Templates:
Find:
Embedded software development templates
Empty Application (C++)
Linux Empty Application
Linux Hello World
Linux Hello World
Let's say 'Hello World' in C.
< Back
Next >
Finish
Cancel

可以看到有三个模板，第一个是 C++应用工程模板，第二个是 Linux 空白应用工程模板，第三个是“Linux Hello World”工程模板。我们选择“Linux Hello World”工程模板，点击底部的“Finish”，进入所示界面：



编译 hello_world 工程。至此，Linux 应用工程的创建就完成了。

5.4 在开发板运行程序

1、将 elf 文件拷贝到 SD 卡中，然后重新启动设备，



系统启动之后，进入到 SD 卡目录下，执行命令可运行程序：

```
sd-mmcblk0p1/ sd-mmcblk0p2/
root@rfevm:~# cd /media/sd-mmcblk0p1/
root@rfevm:/media/sd-mmcblk0p1#
root@rfevm:/media/sd-mmcblk0p1#
root@rfevm:/media/sd-mmcblk0p1#
root@rfevm:/media/sd-mmcblk0p1# ls
BOOT.BIN                    image.ub
System Volume Information   rfevm_app.elf
boot.scr
root@rfevm:/media/sd-mmcblk0p1# ./rfevm_app.elf
Hello World
root@rfevm:/media/sd-mmcblk0p1#
```

2、使用 fileZilla 方式

首先下载 fileZilla 安装包，然后在 windows 下安装：



下载完成之后直接安装即可。

然后进行连接开发板，需要用网线将开发板和电脑连接起来，并且要处于统一网段也就是说电脑能 ping 通开发板：



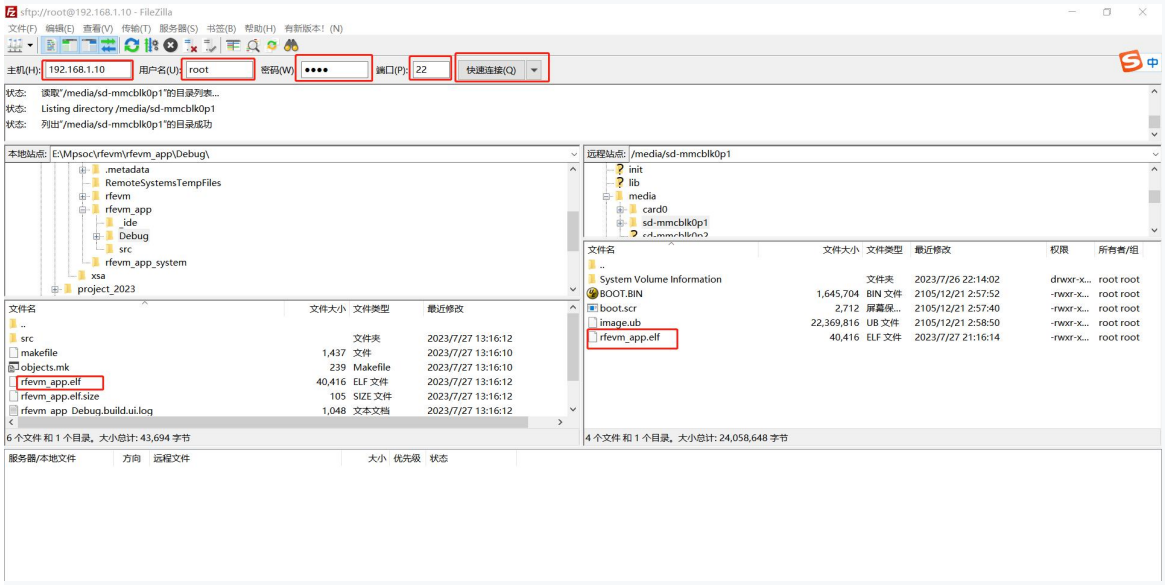
然后打开 fileZilla 工具，并以此填入：

主机地址：开发板地址：192.168.1.10

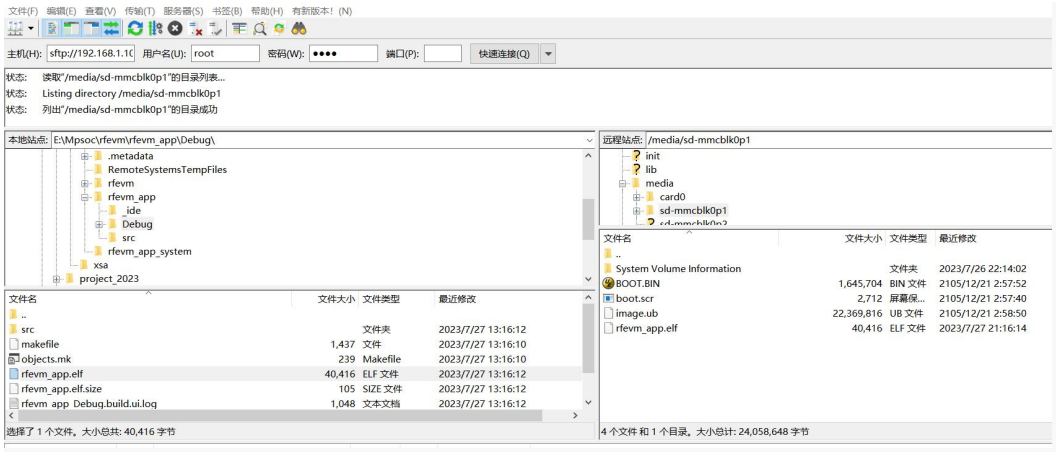
用户名：root

密码：root

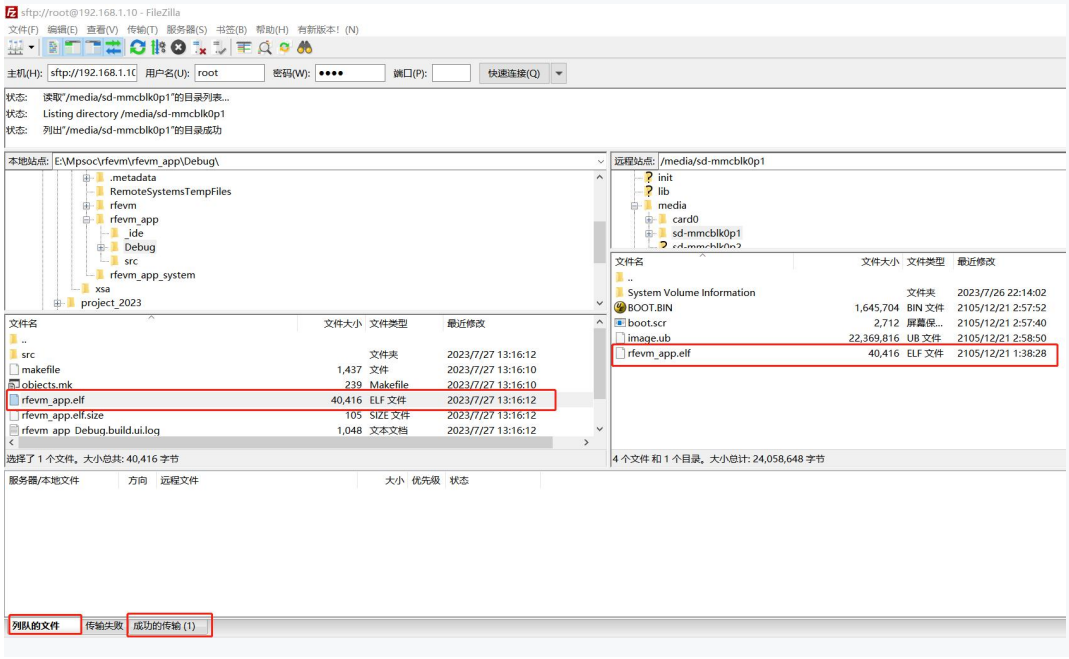
端口：22



填写完成之后，点击快速连接。

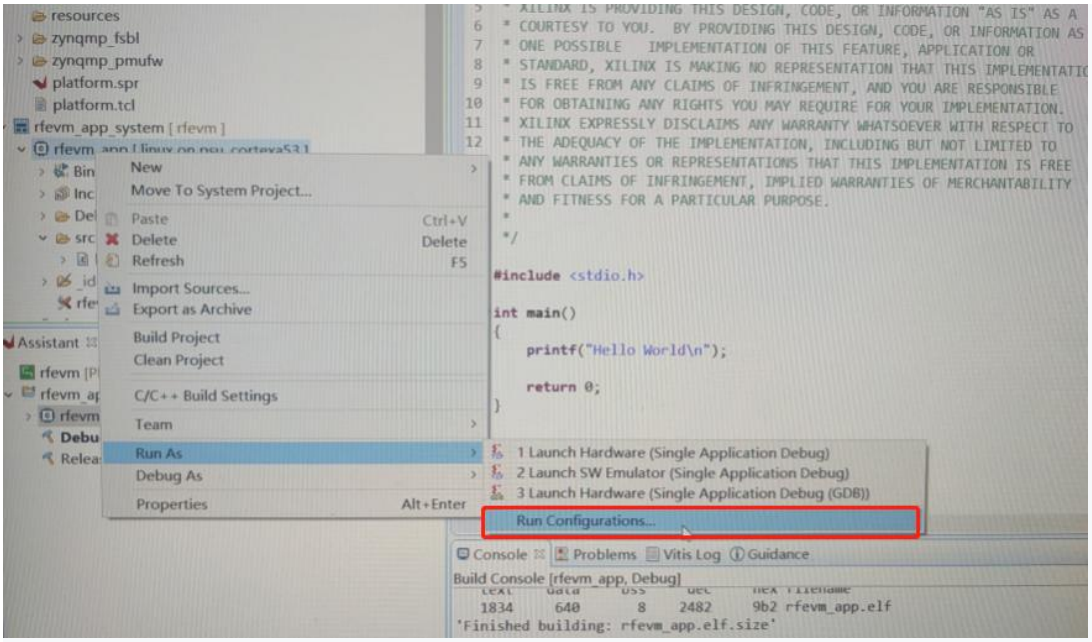


连接之后就可以将电脑上的文件传输到开发板中，右击文件然后点击上传即可。

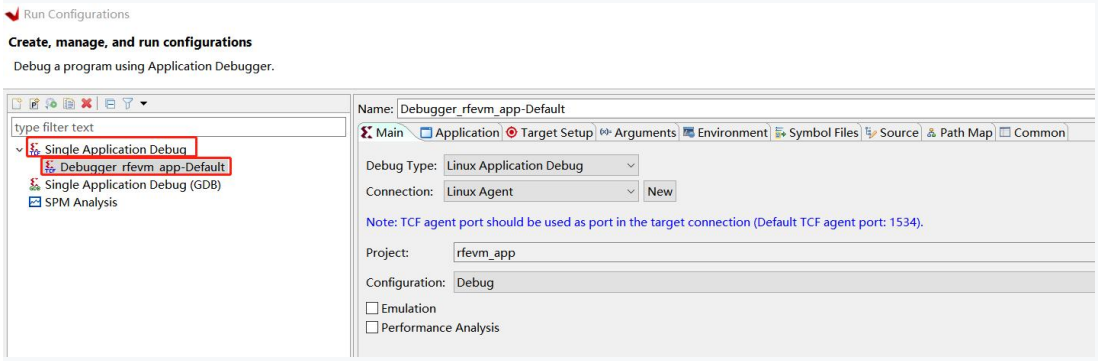


5.5 使用 TCF Agent 方式运行

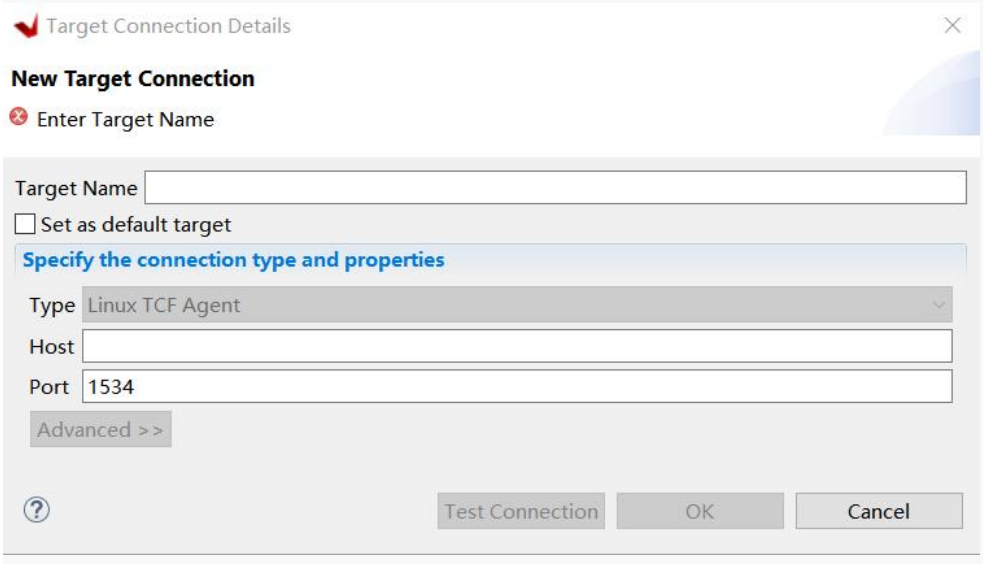
1、在 Vitis 界面，右键点击“rfevm_app”应用工程，在弹出的菜单中选择“Run As—>Run Configuration”



在弹出的界面中， 双击“Single Applicatuin Debug” ，会自动创建“Debugger_rfevm_app-Default”，如下图所示：

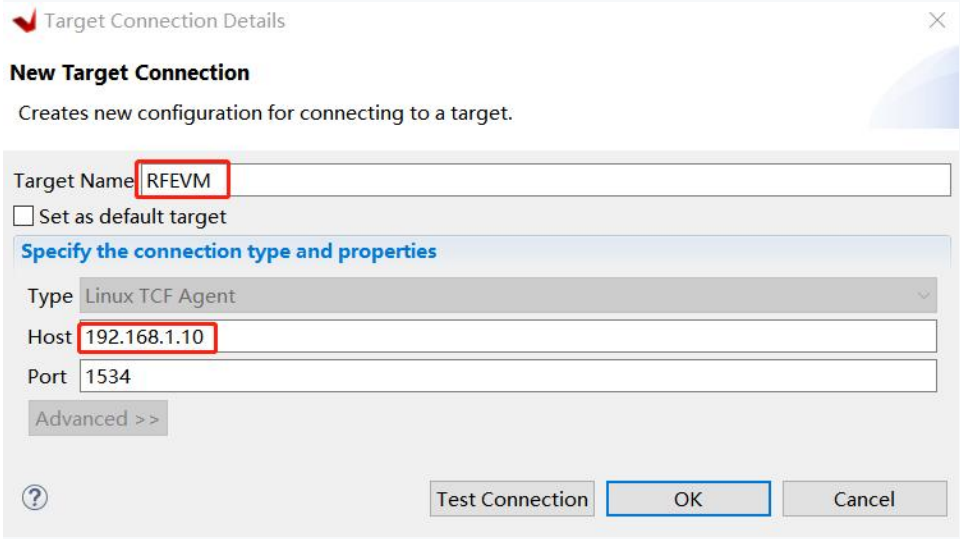


然后点击“New”：

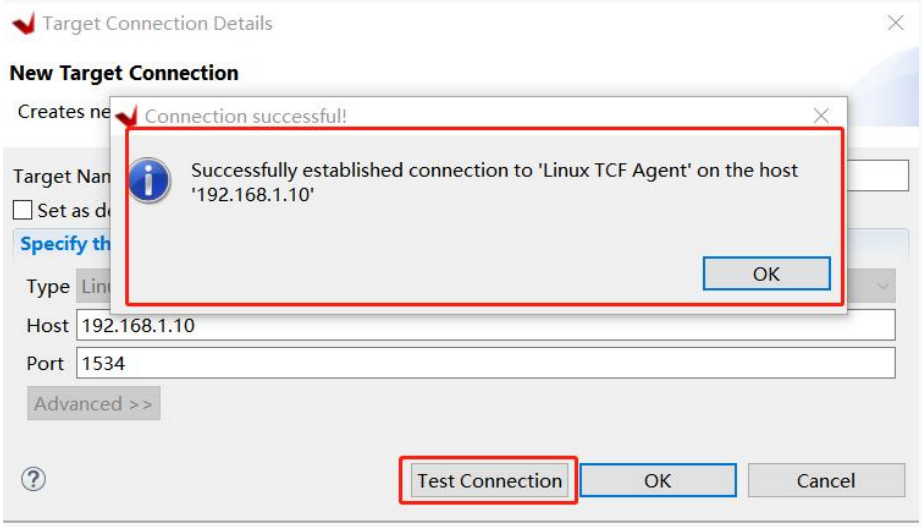


输入 Target Name：RFEVM

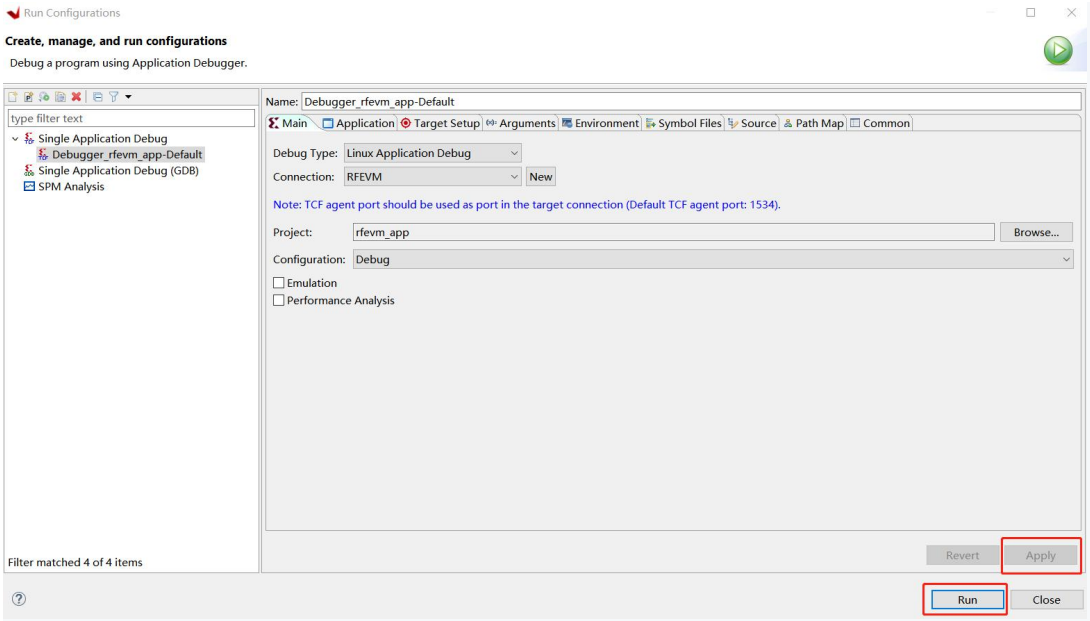
输入 Host 的 IP 地址，也就是开发板的 IP 地址：192.168.1.10



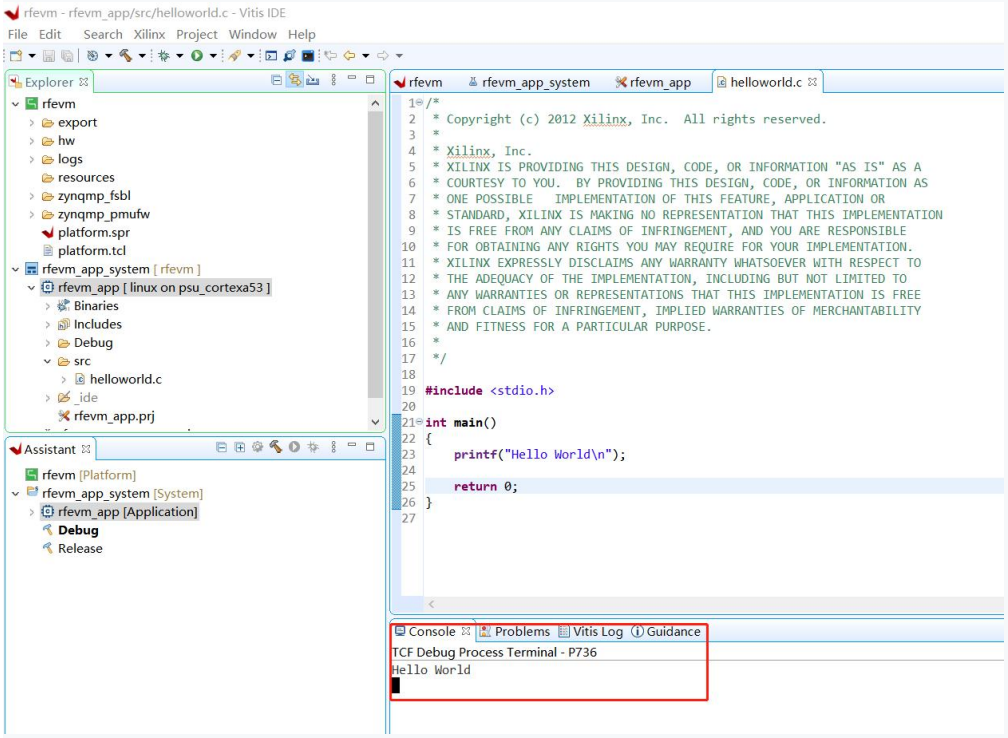
输入 IP 地址完成后，点击“Test Connection”，如果出现下图所示的结果，表示连接测试成功，Linux TCF Agent 服务运行正常，可以与主机（开发板）连接。点击“OK”按钮，完成设置。



完成之后，我们点击 “Apply”，如下图所示：



然后点击“Run”，在 Console 终端可以看到 Hello World 输出，表明程序在 Linux 上运行成功，如下图所示：



6、SD 卡根文件系统

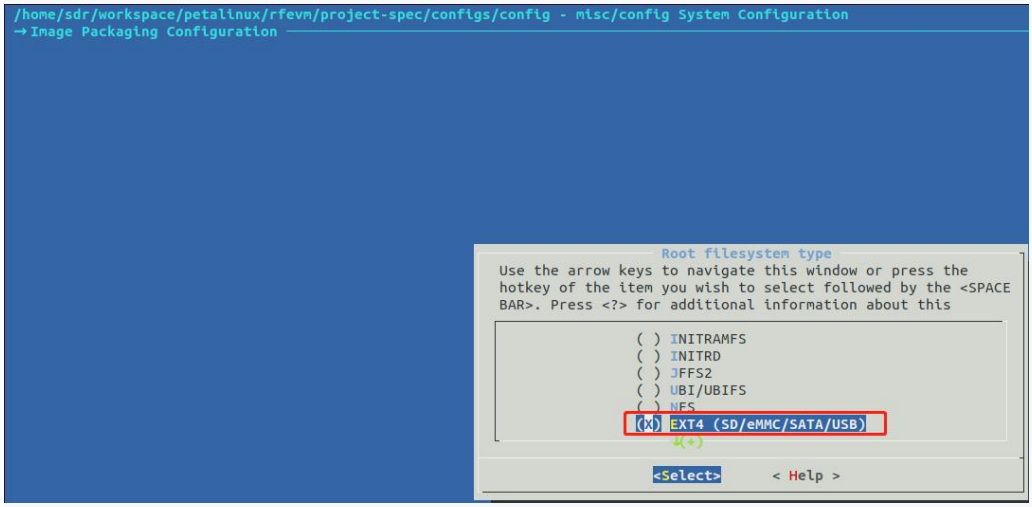
Petalinux 默认的根文件系统类型是 INITRAMFS， 不能保存文件和配置， 本章介绍如何使用 sd 卡来做根文件系统。

6.1 Petalinux 配置

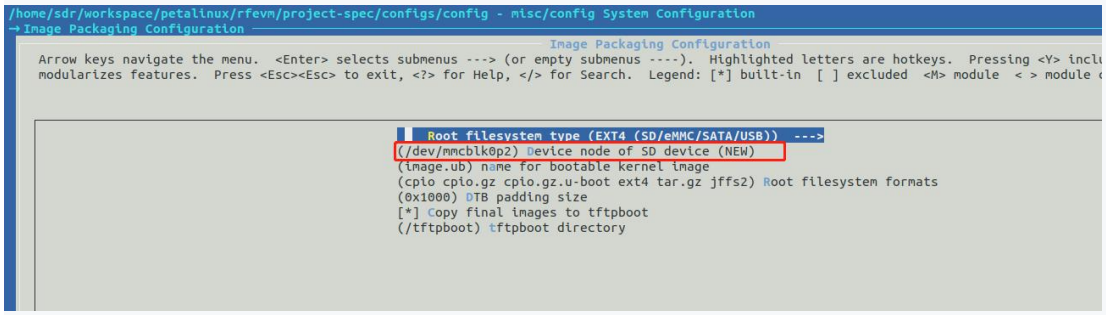
1、运行命令配置 Petalinux

petalinux-config

2、Image Packaging Configuration ->Root filesystem type 选择 (EXT4)



3、根文件系统路径选择至关重要，如果填错无法启动 Linux， mmcblk 代表 sd 卡和 emmc，mmcblk0p2 代表第一个 mmc 设备第二分区， 如果 emmc 不存在，sd 卡存在， 则 sd 为 mmcblk0，所以这里填写/dev/mmcblk0p2



4、然后重新编译工程：patalinux-build

并重新生成启动镜像。

6.2 SD 卡分区制作

1、将 TF 卡插入读卡器，然后将读卡器插入电脑：

2、输入 df -h 查看分区，来查看文件系统使用情况，可以查看挂载点，也可以查看设备节点。

```
sdr@ubuntu:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            1.9G     0  1.9G   0% /dev
tmpfs           393M  6.2M  386M   2% /run
/dev/sda1        55G   25G   29G  47% /
tmpfs           2.0G  252K  2.0G   1% /dev/shm
tmpfs           5.0M  4.0K  5.0M   1% /run/lock
tmpfs           2.0G     0  2.0G   0% /sys/fs/cgroup
tmpfs           393M   56K  393M   1% /run/user/1000
/dev/sdb1       7.5G   45M  7.5G   1% /media/sdr/boot
```

3、卸载：umount /media/sdr/boot，再次查看已经没有 TF 卡设备了，此时就可以使用 fdisk 工具制作了。

4、sudo fdisk /dev/sdb，sdb 要根据 TF 卡在电脑的显示来写，后面不跟数字。

```
sdr@ubuntu:~$ sudo fdisk /dev/sdb

Welcome to fdisk (util-linux 2.27.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.


Command (m for help):
```

5、查看分区：p

```
Command (m for help): p
Disk /dev/sdb: 7.5 GiB, 8053063680 bytes, 15728640 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x55ec5dde

Device     Boot Start      End  Sectors  Size Id Type
/dev/sdb1        2048 15728639 15726592  7.5G  b W95 FAT32

Command (m for help):
```

6、删除分区：d，若有分区，就将分区删除，删除之后在查看分区就没有分区了，若还有就继续删除；直到如下图所示：

```
Command (m for help): p
Disk /dev/sdb: 7.5 GiB, 8053063680 bytes, 15728640 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x55ec5dde

Command (m for help):
```

7、增加第一个分区：n

①增加主分区：p

②分区号：1

③起始地址：默认，回车

④分区大小：+1000M

```
Command (m for help): n
Partition type
   p   primary (0 primary, 0 extended, 4 free)
   e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-15728639, default 2048): 起始扇区，默认，直接回车
Last sector, +sectors or +size{K,M,G,T,P} (2048-15728639, default 15728639): +1000M

Created a new partition 1 of type 'Linux' and of size 1000 MiB.

Command (m for help):
```

8、增加第二个分区：n

①增加主分区：p

②分区号：2

③起始地址：默认回车

④分区大小：默认回车，因为只制作两个分区，剩下的全部给第二个分区。

9、查看分区：p，可以看到已经有两个分区:

```
Command (m for help): p
Disk /dev/sdb: 7.5 GiB, 8053063680 bytes, 15728640 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x55ec5dde

Device     Boot    Start      End  Sectors  Size Id Type
/dev/sdb1           2048  2050047  2048000 1000M 83 Linux
/dev/sdb2        2050048 15728639 13678592  6.5G 83 Linux
```

10、设置分区类型：t，输入大写 L 可以查看所有类型：

①选择分区号：1

②设置为：c

```
Command (m for help): t
Partition number (1,2, default 2): 1
Partition type (type L to list all types): L

 0  Empty           24  NEC DOS         81  Minix / old Lin bf  Solaris
 1  FAT12           27  Hidden NTFS Win 82  Linux swap / So c1  DRDOS/sec (FAT-
 2  XENIX root      39  Plan 9          83  Linux           c4  DRDOS/sec (FAT-
 3  XENIX usr       3c  PartitionMagic  84  OS/2 hidden or  c6  DRDOS/sec (FAT-
 4  FAT16 <32M      40  Venix 80286     85  Linux extended  c7  Syrinx
 5  Extended        41  PPC PReP Boot   86  NTFS volume set da  Non-FS data
 6  FAT16           42  SFS             87  NTFS volume set db  CP/M / CTOS / .
 7  HPFS/NTFS/exFAT 4d  QNX4.x          88  Linux plaintext de  Dell Utility
 8  AIX             4e  QNX4.x 2nd part 8e  Linux LVM       df  BootIt
 9  AIX bootable    4f  QNX4.x 3rd part 93  Amoeba          e1  DOS access
 a  OS/2 Boot Manag 50  OnTrack DM      94  Amoeba BBT      e3  DOS R/O
 b  W95 FAT32       51  OnTrack DM6 Aux 9f  BSD/OS          e4  SpeedStor
 c  W95 FAT32 (LBA) 52  CP/M            a0  IBM Thinkpad hi ea  Rufus alignment
 e  W95 FAT16 (LBA) 53  OnTrack DM6 Aux a5  FreeBSD         eb  BeOS fs
 f  W95 Ext'd (LBA) 54  OnTrackDM6      a6  OpenBSD         ee  GPT
10  OPUS            55  EZ-Drive        a7  NeXTSTEP        ef  EFI (FAT-12/16/
11  Hidden FAT12    56  Golden Bow      a8  Darwin UFS      f0  Linux/PA-RISC b
12  Compaq diagnost 5c  Priam Edisk     a9  NetBSD          f1  SpeedStor
14  Hidden FAT16 <3 61  SpeedStor       ab  Darwin boot     f4  SpeedStor
16  Hidden FAT16    63  GNU HURD or Sys af  HFS / HFS+      f2  DOS secondary
17  Hidden HPFS/NTF 64  Novell Netware  b7  BSDI fs         fb  VMware VMFS
18  AST SmartSleep  65  Novell Netware  b8  BSDI swap       fc  VMware VMKCORE
1b  Hidden W95 FAT3 70  DiskSecure Mult bb  Boot Wizard hid fd  Linux raid auto
1c  Hidden W95 FAT3 75  PC/IX           bc  Acronis FAT32 L fe  LANstep
1e  Hidden W95 FAT1 80  Old Minix       be  Solaris boot    ff  BBT
Partition type (type L to list all types): b

Changed type of partition 'Linux' to 'W95 FAT32'.

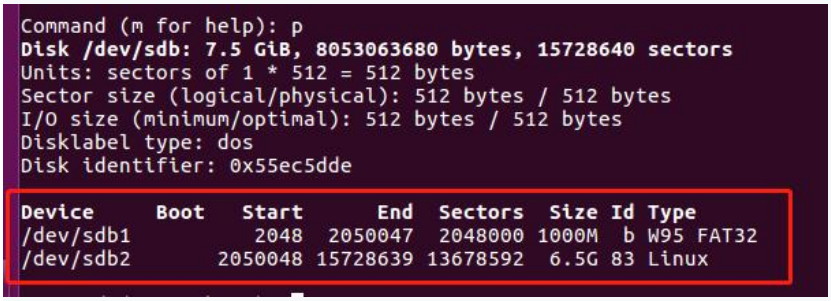
Command (m for help):
```

11、再次设置第二个分区类型：t

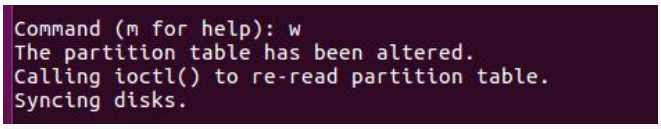
①选择分区号：2

②设置为：83

12、再次查看：p，已经修改完成



13、保存退出：w

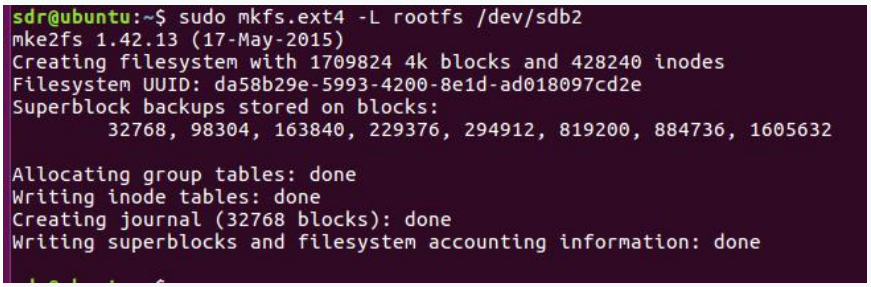


14、分区完成之后，需要格式化分区：

格式化第一个分区：sudo mkfs.vfat -F 32 -n boot /dev/sdb1



格式化第二个分区：sudo mkfs.ext4 -L rootfs /dev/sdb2



15、格式化完成后，我们要去挂载 TF 卡，挂载 TF 卡是为了像文件夹一样操作 TF 卡，将文件拷贝到 TF 卡中。挂载可以使用挂载命令，也可以直接点击 TF 卡 U 盘图标。

```
sdr@ubuntu:~$ sudo mount /dev/sdb1 /media/sdr/boot
sdr@ubuntu:~$ sudo mount /dev/sdb2 /media/sdr/rootfs
```

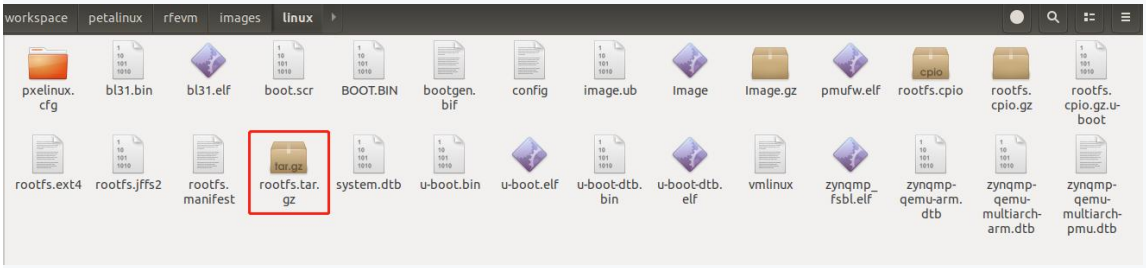
然后可以用 df -h 命令查看挂载点，然后将镜像拷贝到挂载点目录。

```
/dev/sdb1       999M  4.0K  999M   1% /media/sdr/boot
/dev/sdb2        14G   24K   13G   1% /media/sdr/rootfs
```

6.3 SD 卡镜像文件拷贝

1、把 BOOT.bin、boot.scr 和 iamge.ub 复制到 sd 的 FAT32 分区（第一分区）中。

2、解压根文件系统到 SD 卡 EXT4 分区（第二分区）中。



输入解压缩命令，解压缩文件系统。解压缩可能需要几分钟的时间。 (/media/sdr/rootfs 为 SD 卡 EXT4 分区的路径，可能会有不同，请根据自己实际情况修改)。

cd ~/workspace/petalinux/rfevm

sudo tar xzvf images/linux/rootfs.tar.gz -C /media/sdr/rootfs

3、解压完成后，在命令窗口输入"sudo sync"进行同步，同步可能需要几分钟的时间。

4、命令执行完成之后，可以将制作好的 SD 卡插入设备启动设备了。

```
Starting haveged: haveged: command socket is listening at fd 3
haveged: haveged starting up


Starting Dropbear SSH server: dropbear.
Starting internet superserver: inetd.
Starting syslogd/klogd: done
Hello PetaLinux World
Starting tcf-agent: [    6.231036] random: crng init done
[    6.234447] random: 7 urandom warning(s) missed due to ratelimiting
OK

root@rfevm:~# [    8.559639] macb ff0e0000.ethernet eth0: Link is Up - 1Gbps/Full - flow control off
[    8.567318] IPv6: ADDRCONF(NETDEV_CHANGE): eth0: link becomes ready

root@rfevm:~#
```

同时你创建的文件在重启后也不会丢失。

7、Linux 下 NVME SSD 操作

7.1 petalinux 配置

1、PCIe 驱动默认是配置好的，需要再配置 NVMe 驱动。 在 petalinux 工程目录下输入：

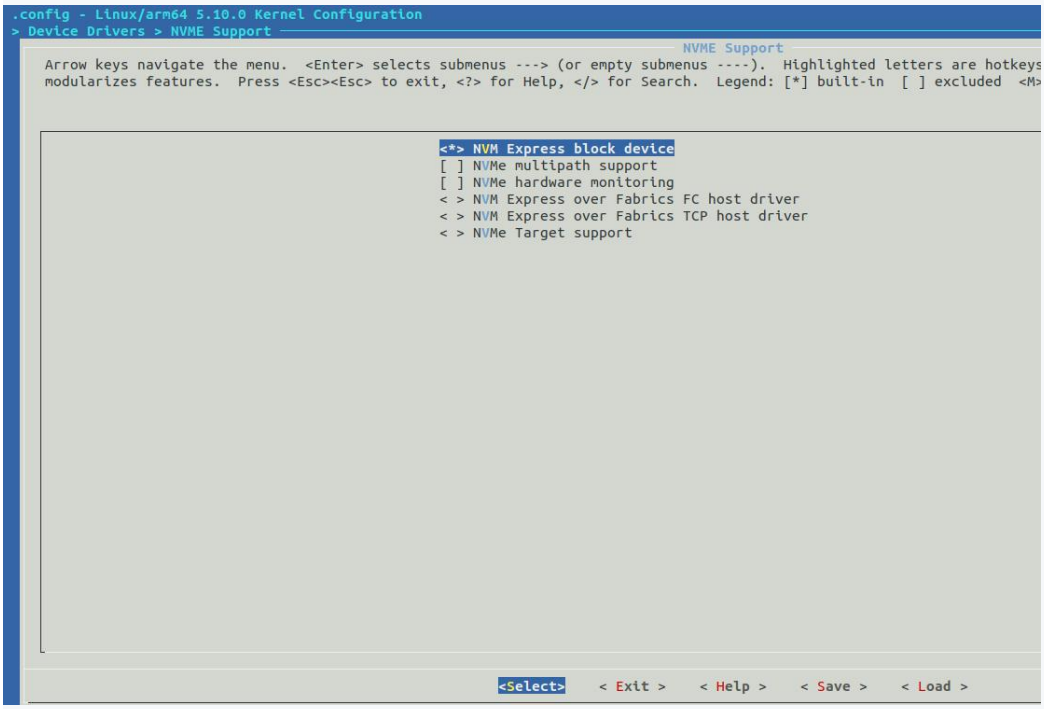
petalinux-config -c kernel

2、在 Device Drivers 中选择<*> NVM Express block device。

Device Drivers

NVME Support

<*> NVM Express block device



3、配置 rootfs，配置带有 PCI 实用程序组件的常用命令：

petalinux-config -c rootfs

启用 PCI utils(用于 lspci)：

→ Filesystem Packages

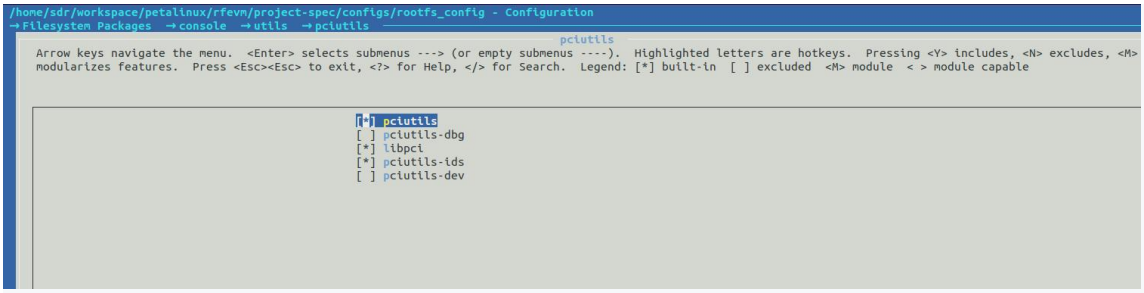
→ console

→ utils

[*] pciutils

[*] libpci

[*] pciutils-ids



启用 lsblk, fdisk, mkfs, blkid 所需的包:

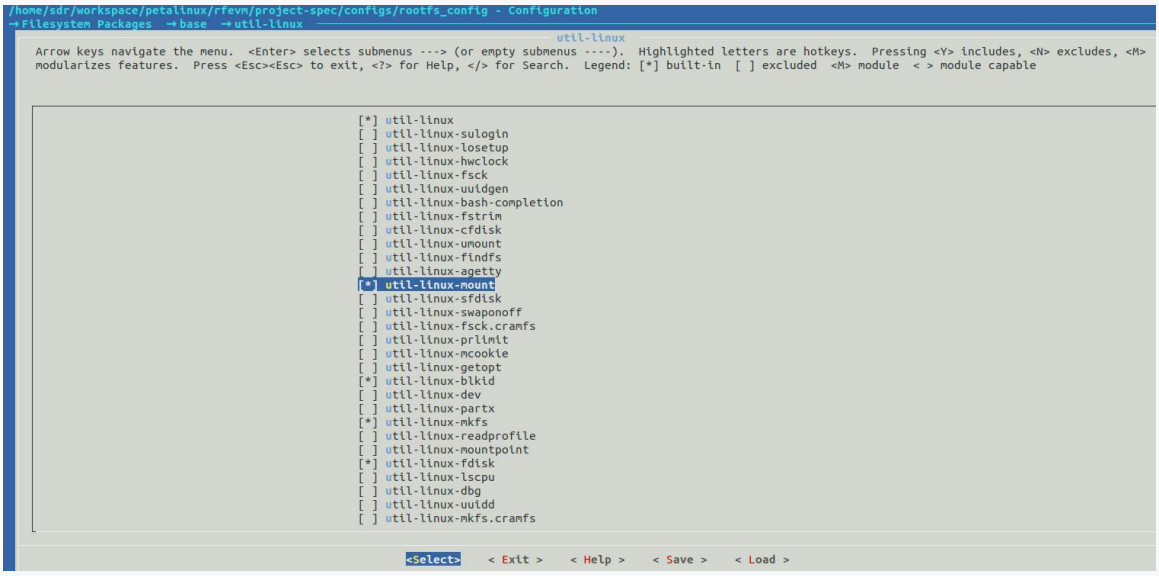
Filesystem Packages->base->util-linux->util-linux

Filesystem Packages->base->util-linux->util-linux-blkid

Filesystem Packages->base->util-linux->util-linux-fdisk

Filesystem Packages->base->util-linux->util-linux-mkfs

Filesystem Packages->base->util-linux->util-linux-mount



4、重新编译工程：petalinux-build

并重新生成启动镜像。

5、内核输出日志

```
[    3.703503] nwl-pcie fd0e0000.pcie: host bridge /axi/pcie@fd0e0000 ranges:
[    3.710398] nwl-pcie fd0e0000.pcie:      MEM 0x00e0000000..0x00efffffff -> 0x00e0000000
[    3.718398] nwl-pcie fd0e0000.pcie:      MEM 0x0600000000..0x07ffffffff -> 0x0600000000
[    3.726461] nwl-pcie fd0e0000.pcie: Link is UP
[    3.731035] nwl-pcie fd0e0000.pcie: PCI host bridge to bus 0000:00
[    3.737212] pci_bus 0000:00: root bus resource [bus 00-ff]
[    3.742695] pci_bus 0000:00: root bus resource [mem 0xe0000000-0xefffffff]
[    3.749560] pci_bus 0000:00: root bus resource [mem 0x600000000-0x7fffffff pref]
[    3.757058] pci 0000:00:00.0: [10ee:d022] type 01 class 0x060400
[    3.763118] pci 0000:00:00.0: PME# supported from D0 D1 D2 D3hot
[    3.770257] pci 0000:01:00.0: [1e0f:0009] type 00 class 0x010802
[    3.776313] pci 0000:01:00.0: reg 0x10: [mem 0x00000000-0x00003fff 64bit]
[    3.783402] pci 0000:01:00.0: 8.000 Gb/s available PCIe bandwidth, limited by 5.0 GT/s PCIe x2 link at 0000:
00:00.0 (capable of 31.504 Gb/s with 8.0 GT/s PCIe x4 link)
[    3.799322] pci 0000:00:00.0: BAR 8: assigned [mem 0xe0000000-0xe00fffff]
[    3.806107] pci 0000:01:00.0: BAR 0: assigned [mem 0xe0000000-0xe0003fff 64bit]
[    3.813435] pci 0000:00:00.0: PCI bridge to [bus 01-0c]
[    3.818656] pci 0000:00:00.0:   bridge window [mem 0xe0000000-0xe00fffff]
[    3.825696] nvme nvme0: pci function 0000:01:00.0
[    3.830424] pci 0000:00:00.0: enabling device (0000 -> 0002)
[    3.830760] xilinx-zynqmp-dma fd500000.dma: ZynqMP DMA driver Probe success
[    3.836081] nvme 0000:01:00.0: enabling device (0000 -> 0002)
[    3.843233] xilinx-zynqmp-dma fd510000.dma: ZynqMP DMA driver Probe success
[    3.851262] nvme nvme0: Shutdown timeout set to 10 seconds
```

若输出以上日志则表明，驱动加载成功。

可用命令查看：

```
root@rfevm:~# ls /dev/nvme0*
/dev/nvme0      /dev/nvme0n1    /dev/nvme0n1p1
root@rfevm:~#
```

```
root@rfevm:~# lspci
00:00.0 PCI bridge: Xilinx Corporation Device d022
01:00.0 Non-Volatile memory controller: KIOXIA Corporation Device 0009 (rev 01)
root@rfevm:~#
```

7.2 读写测速

1、顺序写测试

time dd if=/dev/zero of=/dev/nvme0n1 bs=2M count=8196

8196+0 records in

8196+0 records out

real 0m38.561s

user 0m0.048s

sys 0m38.401s

2、顺序读测试， 测试之前注意清理缓存。

因为 LINUX 的内核机制，一般情况下不需要特意去释放已经使用的 cache。这些 cache 内容可以增加文件以及的读写速度。

执行下面指令清除缓存

```
echo 3 > /proc/sys/vm/drop_caches

time dd if=/dev/nvme0n1 of=/dev/null bs=2M count=8196
8196+0 records in
8196+0 records out

real    0m35.374s
user    0m0.004s
sys     0m26.612s
```

8、USB 的使用

USB 是很常用的接口，目前大多数的设备都是 USB 接口的，比如鼠标、键盘、USB 摄像头等，我们在实际开发中也常常遇到 USB 接口的设备。USB 外设的使用很简单，一些常用的 USB 外设如鼠标、键盘、U 盘可以直接插到开发的 USB 接口上，像插入电脑的 USB 接口一样。

8.1 USB 接口简介

1、usb 设备接入主机后，匹配过程如下：

1) 硬件检测

usb 接口有四条线分别为 5V、GND、D-、D+。主机端 usb 接口中的 D-戒 D+有下拉电阻，空载时为低电平。设备端的 usb 接口内部 D-或 D+有上拉电阻，usb 设备接入主机后，会把主机 usb 接口的 D-或 D+拉高，从而主机从硬件的角度检测到 usb 设备的接入。

2) 握手匹配

主机检测到 usb 设备接入后，就会和设备进行交互，主机端主动发起获取设备信息的描述符，设备则需要按照固定格式返回描述符。如果实在 windows 桌面系统下，此时我们就能看到 xxx 设备接入的弹窗了。

3) 分配地址

一个主机上可以接入多个 usb 讴备，为了区分这些设备，在握手成功后，主机会给设备分配设备地址，主机发出的命令中都会包含地址信息。在握手之前，主机会用编号 0 来和设备交互。

8.2 USB 传输

1、usb 的传输时主从结极，主就是主机，从就是设备。所有的传输都是由主机发起的，设备没有主动通知主机的能力。我们常说的输入输出，都是在主机的角度去描述的，比如输入设备键盘、鼠标，输出设备显示器等等。

usb 的传输模式，在主从结极的基础上，usb 可以分为四种传输模式：

1) 控制传输

控制传输是一种特殊的传输方式，相对复杂一些。usb 设备连接主机时，主机使用控制传输方式发送控制命令对设备进行配置。同时，需要通过控制传输获取 usb 设备的描述符对设备进行识别，在设备的枚举过程中都是使用控制传输进行数据交换。可靠，且时效性可保证。

2) 批量传输

批量传输一般用于数据量大但对时间要求又不高的场合，如 u 盘、打印机等。可靠，不保证时效性。

3) 中断传输

中断传输一般用于数据量小，不连续且实时要求搞得场合，如鼠标、键盘等输入设备。

4) 等时传输

等时传输一般用于数据量大、连续且实时要求高的场合，但可靠性难以保证，一般用于摄像头、话筒等设备。

8.3 USB 总线驱动

1、usb 总线驱动

总线驱动已经不陌生了，常用的 palform 驱动和 i2c 驱动都是这种思想。同样 usb 也是分为总线和设备两块驱动。

usb 总线驱动和硬件对接，起到一个承接作用。前面说的 usb 的初始化过程，就需要总线驱动来完成。总线驱动需要做的事有这些：

1) 识别 usb 设备

2) 查找并安装相应的设备驱动

3) 为设备驱动提供读写函数

总线驱动一般会由芯片厂家提供，少做了解即可。

8.4 USB 设备驱动

1、usb 设备驱动中，同样是分成了设备和驱动两个部分。与 i2c 等有区别的地方是，usb 只需要的去实现驱动部分，设备部分由总线驱动获取。

当接上一个 USB 设备时，就会产生中断 hub_irq(),在中断里会分配一个编号地址 choose_address(udev)。总线把这个地址告诉设备 hub_set_address()。接着发出命令获取设备描述符 usb_get_device_descriptor()。获取到设备信息后，会注册一个设备 device_add()。这个 device 会被放到 usb 总线 usb_bus_type 的设备链表中。usb 驱动程序注册后会从 usb 总线的驱动链表中取出设备驱动结构 usb_driver，把 usb_interface 和 usb_driver 的 id_table 比较，匹配成功就会调用 usb_driver 中的 probe 函数。

而我们要做的就是定义初始化并注册 usb_driver，并实现 probe 函数。

先来看看 usb_driver 结构体，定义在文件 include/linux/usb.h 中如下：

```
1. struct usb_driver {
2.              const char *name;
3.
4.              int (*probe) (struct usb_interface *intf,
5.              const struct usb_device_id *id);
6.
7.              void (*disconnect) (struct usb_interface *intf);
8.
9.              int (*unlocked_ioctl) (struct usb_interface *intf, unsigned int code,
10.             void *buf);
11.
12.             int (*suspend) (struct usb_interface *intf, pm_message_t message);
13.             int (*resume) (struct usb_interface *intf);
14.             int (*reset_resume)(struct usb_interface *intf);
15.
```

```
16.             int (*pre_reset)(struct usb_interface *intf);
17.             int (*post_reset)(struct usb_interface *intf);
18.
19.             const struct usb_device_id *id_table;
20.
21.             struct usb_dynids dynids;
22.             struct usbdrv_wrap drvwrap;
23.             unsigned int no_dynamic_id:1;
24.             unsigned int supports_autosuspend:1;
25.             unsigned int disable_hub_initiated_lpm:1;
26.             unsigned int soft_unbind:1;
27. };
```

name 为设备名。

probe 函数就是设备匹配成功后会执行的函数，必项实现。

disconnect 函数在设备不可用时会执行，如设备拔出。

id_table 用于匹配设备。usb_device_id 定义在 include/linux/mod_devicetable.h 中，何以使用下面的宏去初始化：

USB_INTERFACE_INFO(cl,sc,pr)

cl 是 class 类，sc 是 sub class 子类，pr 是指协议。这些设备描述符定义在文件 include\linux\usb\Ch9.h 中。总线驱动会根据这些描述符去匹配设备和驱动。

定义并初始化好 usb_driver 后，使用下面的宏向内核注册：

usb_register(driver)

相对的使用下面的函数注销：

void usb_deregister(struct usb_driver *);

usb 设备驱动示例：

```
1. static struct usb_device_id usb_id_table [] =
2. {
3. { USB_INTERFACE_INFO(XXX, XXX, XXX) },
4. { }
5. };
6.
```

```
7.
8. static int usb_probe(struct usb_interface *intf, const struct usb_device_id *id)
9. {
10. return 0;
11. }
12.
13. static void usb_disconnect(struct usb_interface *intf)
14. {
15.
16. }
17.
18. static struct usb_driver usb_driver = {
19. .name = "xxx",
20. .probe = usbmouse_as_key_probe,
21. .disconnect = usbmouse_as_key_disconnect,
22. .id_table = usbmouse_as_key_id_table,
23. };
24.
25.
26. static int usb_init(void)
27. {
28. usb_register(&usb_driver);
29. return 0;
30. }
31.
32. static void usb_exit(void)
33. {
34. usb_deregister(&usb_driver);
35. }
36.
37. module_init(usb_init);
38. module_exit(usb_exit);
39.
40. MODULE_LICENSE("GPL");
```

probe 函数和 disconnect 函数的输入参数 struct usb_interface *intf 可以使用和函数 interface_to_usbdev()来获取 usb_device，如

struct usb_device *dev = interface_to_usbdev(intf);

8.5 URB 请求块

1、urb 是设备驱动中用来描述 usb 设备通信的基本数据结构，是端点处理的对象，操作 usb 设备需要使用 urb 来进行。urb 定义在文件 include/linux/usb.h 中，他的定义和注释如下：

```
1. struct urb {
2. /* 私有的：只能由 USB 核心和主机控制器访问的字段 */
3. struct kref kref; /*urb 引用计数 */
4. void *hcpriv; /* 主机控制器私有数据 */
5. atomic_t use_count; /* 并发传输计数 */
6. u8 reject; /* 传输将失败*/
7. int unlink; /* unlink 错误码 */
8. /* 公共的： 可以被驱动使用的字段 */
9. struct list_head urb_list; /* 链表头*/
10. struct usb_anchor *anchor;
11. struct usb_device *dev; /* 关联的 USB 设备 */
12. struct usb_host_endpoint *ep;
13. unsigned int pipe; /* 管道信息 */
14. int status; /* URB 的当前状态 */
15. unsigned int transfer_flags; /* URB_SHORT_NOT_OK | ...*/
16. void *transfer_buffer; /* 发送数据到设备或从设备接收数据的缓冲区 */
17. dma_addr_t transfer_dma; /*用来以 DMA 方式向设备传输数据的缓冲区 */
18. int transfer_buffer_length;/*transfer_buffer 或 transfer_dma 指向缓冲区的大小 */
19.
20. int actual_length; /* URB 结束后，发送或接收数据的实际长度 */
21. unsigned char *setup_packet; /* 指向控制 URB 的设置数据包的指针*/
22. dma_addr_t setup_dma; /*控制 URB 的设置数据包的 DMA 缓冲区*/
```

```
23. int start_frame; /*等时传输中用于设置或返回初始帧*/
24. int number_of_packets; /*等时传输中等时缓冲区数量 */
25. int interval; /* URB 被轮询到的时间间隔（对中断和等时 urb 有效） */
26. int error_count; /* 等时传输错误数量 */
27. void *context; /* completion 函数上下文 */
28. usb_complete_t complete; /* 当 URB 被完全传输或发生错误时，被调用 */
29. /*单个 URB 一次可定义多个等时传输时，描述各个等时传输 */
30. struct usb_iso_packet_descriptor iso_frame_desc[0];
31. };
```

urb 使用流程如下：

1、 创建 urb

使用下面的函数创建一个 urb：

```
struct urb *usb_alloc_urb(int iso_packets, int mem_flags);
```

相对的使用下面的函数释放 urb：

```
void usb_free_urb(struct urb *urb);
```

2、 填充 urb

1) 对于中断 urb，使用下面的函数来初始化：

```
void usb_fill_int_urb(struct urb *urb, struct usb_device *dev, unsigned int pipe,
void *transfer_buffer, int buffer_length, usb_complete_t complete, void *context,
int interval);
```

参数说明：

urb：要被初始化的 urb 的指针；

dev：该 urb 要被发送到的 usb 设备；

pipe：该 urb 要被发送到的 usb 设备的特定端点，使用 usb_sndctrlpipe()或 usb_rcvictrlpipe()函数来创建；

transfer_buffer：发送数据或接收数据的缓冲区的指针。它也不能是静态缓冲区，必项使用 kmalloc()来分配；

buffer_length：transfer_buffer 指针所指向缓冲区的大小；

complete：该 urb 完成时的回掉函数；

context：完成处理函数的上下文；

interval：该这个 urb 调度的间隔。

2) 对于批量 urb，使用下面的函数来初始化：

void usb_fill_bulk_urb(struct urb *urb, struct usb_device *dev,unsigned int pipe,
void *transfer_buffer,int buffer_length, usb_complete_t complete,void *context);

这里的 pipe 需要使用 usb_sndbulkpipe()或者 usb_rcvbulkpipe()函数来创建。

其他参数与 usb_fill_int_urb()相同。

3) 对于控制 urb，使用下面的函数来初始化：

void usb_fill_control_urb(struct urb *urb, struct usb_device *dev,unsigned int pipe,
unsigned char *setup_packet,void *transfer_buffer, int
buffer_length,usb_complete_t complete, void *context);

setup_packet：即将被发送到端点的设置数据包。

这里的 pipe 需要使用 usb_sndctrlpipe()或 usb_rcvictrlpipe()函数来创建。

其他参数与 usb_fill_int_urb()相同。

3、 提交 urb

完成填充后，使用下面的函数来提交 urb：

int usb_submit_urb(struct urb *urb, int mem_flags);

mem_flags 有以下定义：

GFP_ATOMIC：在中断服务函数、底半部、tasklet、定时器处理函数以及 urb 完成回掉函数中，如果调用者持有自旋锁或者读写锁时以及当驱动将当前进程修改为非 TASK_RUNNING 时使用。

GFP_NOIO：在存储设备的块 I/O 和错误处理路经中使用；

GFP_KERNEL：其他情况都使用这个标志。

在 urb 提交后，如果 urb 成功发送给设备、数据发送发生错误或驱动使用 usb_unlink_urb()或 usb_kill_urb()主动取消 urb 时，urb 会结束。urb 结束时可以通过成员发量 status 来查看结束的原因。

8.6 USB 测试

1、Petalinux 已经提供了 USB 驱动，我们只需要添加对应的设备树信息，即可实现 USB 驱动。

打开开发板，接上 U 盘后，可以在控制台中看到识别到 U 盘的相关信息。

```
root@rfevm:/media/card0# [  373.331939] usb 2-1: new SuperSpeed Gen 1 USB device number 2 using xhci-hcd
[  374.764950] usb 2-1: New USB device found, idVendor=1f75, idProduct=0917, bcdDevice= 0.01
[  374.773121] usb 2-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[  374.780253] usb 2-1: Product: PenDrive
[  374.784001] usb 2-1: Manufacturer: Innostor
[  374.788176] usb 2-1: SerialNumber: 3404081559
[  374.793150] usb-storage 2-1:1.0: USB Mass Storage device detected
[  374.799584] scsi host0: usb-storage 2-1:1.0
[  375.822468] scsi 0:0:0:0: Direct-Access     Innostor Innostor        1.00 PQ: 0 ANSI: 6
[  375.832306] sd 0:0:0:0: [sda] 61440000 512-byte logical blocks: (31.5 GB/29.3 GiB)
[  375.840084] sd 0:0:0:0: [sda] Write Protect is off
[  375.845003] sd 0:0:0:0: [sda] Write cache: disabled, read cache: disabled, doesn't support DPO or FUA
[  375.871483]  sda: sda4
[  375.876428] sd 0:0:0:0: [sda] Attached SCSI removable disk

root@rfevm:/media/card0#
```

拔掉 U 盘后显示：

```
root@rfevm:/media/card0#
root@rfevm:/media/card0# [  431.627895] usb 2-1: USB disconnect, device number 2

root@rfevm:/media/card0#
```

9、以太网的使用

首先，嵌入式网络硬件分为两部分：MAC 和 PHY，一般都是通过看数据手册来判断一款 SoC 是否支持网络，如果一款芯片数据手册说自己支持网络，一般都是说的这款 SoC 内置 MAC，MAC 类似 I2C 控制器、SPI 控制器一样的外设。

但是光有 MAC 还不能直接驱动网络，还需要另外一个芯片：PHY，因此对于内置 MAC 的 SoC，其外部必须搭配一个 PHY 芯片。但是有些 SoC 内部没有 MAC，那也就没法搭配 PHY 芯片了，这些内部没有网络 MAC 的芯片如何上网呢？这里就要牵扯出常见的两个嵌入式网络硬件方案了。

1、SOC 内部没有网络 MAC 外设

我们一般说某个 SoC 不支持网络，说的就是它没有网络 MAC。那么这个芯片就不能上网了吗？显然不是的，既然没有内部 MAC，那么可以找个外置的 MAC 芯片啊，不过一般这种外置的网络芯片都是 MAC+PHY 一体的。这种方案的优点就是让不支持网络的 SoC 能够另辟蹊径，实现网络功能，但是缺点就是网络效率不高，而且此类芯片网速都不快。

2、SOC 内部集成网络 MAC 外设

我们一般说某个 SoC 支持网络，说的就是他内部集成网络 MAC 外设，此时我们还需要外接一个网络 PHY 芯片。此时就有朋友会有疑问，PHY 芯片不能也集成进 SoC 吗？笔者目前还没见过将 PHY 也集成到芯片里面的 SoC。

一般常见的通用 SoC 都会集成网络 MAC 外设，比如 STM32F4/F7/H7 系列、NXP 的 I.MX 系列以及 ZYNQ，内部集成网络 MAC 的优点如下：

①、内部 MAC 外设会有专用的加速模块，比如专用的 DMA，加速网速数据的处理。

②、网速快，可以支持 10/100/1000M 网速。

③、外接 PHY 可选择性多，成本低。

内部的 MAC 外设会通过相应的接口来连接外部 PHY 芯片，根据数据传输模式不同，大致可以分为以下两类：

- MII/RMII 接口：支持 10Mbit/s 和 100Mbit/s 数据传输模式；
- GMII/RGMII 接口：支持 10Mbit/s、100Mbit/s 以及 1000Mbit/s 数据传输模式。

3、SOC 含有网络 MAC 外设，因此网络驱动主要就是网络 MAC 外设的驱动。

4、网络外设 MAC 驱动以及 PHY 芯片驱动，网络外设 MAC 驱动一般 SoC 厂商会提供，譬如我们所使用的内核源码中就已经包含了网络外设 MAC 的驱动源码，通过网络设备节点的 compatible 属性值就可以找到；PHY 芯片驱动一般会有通用驱动文件，还有 PHY 芯片厂商针对自己的芯片编写对应的 PHY 驱动，大部分常见的 PHY 在我们的内核源码中都可以找到对应的驱动源码，总体来说，SoC 内置网络 MAC+外置 PHY 芯片这种方案我们是不需要编写什么驱动的，基本可以直接使用。

9.1 linux 驱动框架

从功能得角度看，网络设备需要实现的功能有三，初始化、发送和接收，我们以 net_device 结构体为切入点，以三个需要实现的功能为引来看 Linux 网卡驱动程序的实现方法。

1、初始化

Linux 内核使用 net_device 结构体表示一个具体的网络设备，net_device 是整个网络驱动的灵魂。网络驱动的核心就是初始化 net_device 结构体中的各个成员变量，然后将初始化完成以后的 net_device 注册到 Linux 内核中。net_device 结构体定义在 include/linux/netdevice.h 中，net_device 是一个庞大的结构体

```
1. struct net_device {
2. char name[IFNAMSIZ]; //网卡设备名称
3. struct hlist_node name_hlist;
4. char *ifalias;
5.
```

```
6. unsigned long mem_end; //该设备的内存结束地址
7. unsigned long mem_start; //该设备的内存起始地址
8. unsigned long base_addr; //该设备的内存 I/O 基地址
9. int irq; //该设备的中断号
10.
11. atomic_t carrier_changes;
12.
13. unsigned long state; //网络设备和网络适配器的状态信息
14.
15. ……
16.
17. struct net_device_stats stats; //用来保存统计信息的 net_device_stats 结构体
18.
19. ……
20.
21. const struct net_device_ops *netdev_ops;
22. //网络设备操作函数
23. ……
24.
25. unsigned int flags; //flags 指网络接口标志,常见取值如下
26. //IFF_UP:设备被激活并可以开始发送数据包时设置该
标志
27. //IFF_AUTOMEDIA:设置设备可在多种媒介间切换
28. //IFF_BROADCAST:允许广播
29. //IFF_DEBUG:调试模式
30. //IFF_LOOPBACK:回环
31. //IFF_NOARP:无可执行 ARP,点对点接口就不需要运行
ARP
32. //IFF_POINTOPOINT:点到点链路
33. ……
34.
```

```
35. unsigned int mtu; //最大数据包
36. unsigned short type; //接口的硬件类型
37. unsigned short hard_header_len;
38. //硬件帧头长度,一般被赋为 ETH_HLEN 值为 14
39.
40. ……
41. unsigned long last_rx; //接收数据包的时间戳,调用 netif_rx()后赋上
jiffies 即可
42. unsigned char *dev_addr; //MAC 地址
43.
44. ……
45. };
```

除了一些基本信息名称、地址之外，最为关键的成员就是 struct net_device_ops 类型的指针变量 netdev_ops，他包含了网络设备的操作函数集我们需要实现的发包含数也包含在其中，看一下他的具体定义，也在这个文件中，如下(有省略)：

```
1. struct net_device_ops {
2. ……
3. int (*ndo_open)(struct net_device *dev);
4. int (*ndo_stop)(struct net_device *dev);
5. netdev_tx_t (*ndo_start_xmit)(struct sk_buff *skb,
6. struct net_device *dev);
7. ……
8. };
```

ndo_open 用于开启网络设备，ndo_stop 用于关闭网络设备。

ndo_start_xmit 函数就是关键的发包函数了。

再看一下 net_device 结构体中的 struct net_device_stats stats 成员，net_device_stats 结构体定义如下：

```
1. struct net_device_stats {
2. unsigned long rx_packets;
```

```
3. unsigned long tx_packets;
4. unsigned long rx_bytes;
5. unsigned long tx_bytes;
6. unsigned long rx_errors;
7. unsigned long tx_errors;
8. unsigned long rx_dropped;
9. unsigned long tx_dropped;
10. unsigned long multicast;
11. unsigned long collisions;
12. unsigned long rx_length_errors;
13. unsigned long rx_over_errors;
14. unsigned long rx_crc_errors;
15. unsigned long rx_frame_errors;
16. unsigned long rx_fifo_errors;
17. unsigned long rx_missed_errors;
18. unsigned long tx_aborted_errors;
19. unsigned long tx_carrier_errors;
20. unsigned long tx_fifo_errors;
21. unsigned long tx_heartbeat_errors;
22. unsigned long tx_window_errors;
23. unsigned long rx_compressed;
24. unsigned long tx_compressed;
25. };
```

实际就是网络状态的统计信息合集，我们会在收发处理函数中去操作这个 stats 来记录状态。

可见 net_device 结构体变量是网卡设备的关键，网卡设备的初始化也就是 net_device 结构体变量的初始化，步骤如下：

1) 定义一个 net_device 结构体变量

2) 使用函数 alloc_netdev()分配一个 net_device 结构体变量

3) 初始化硬件寄存器

4) 设置 net_device 结构体变量成员发量

5) 使用 register_netdev()注册 net_device 结构体变量

alloc_netdev()是个宏定义，具体如下：

alloc_netdev(sizeof_priv, name, name_assign_type, setup)

第一个参数 sizeof_priv 为私有变量的大小。

第二个参数 name 为设备名。

第三个参数 name_assign_type 为设备来源名称。

第四个参数 setup 为 net_device 的 setup()函数指针，setup()函数接收的参数为 struct net_device 指针，用于预置 net_device 成员的值。

register_netdev()只有一个输入参数就是需要注册的 net_device 结构体变量指针。

2、发包

发包函数 ndo_start_xmit 需要我们自己构建，原型如下：

netdev_tx_t (*ndo_start_xmit)(struct sk_buff *skb, struct net_device *dev);

第一个输入参数 struct sk_buff 是另 socket(套接字)缓冲区，用于网络模型中各层之间的数据传递。struct sk_buff 定义在文件 include/linux/skbuff.h 中，是一个双向链表，如下(有省略)：

```
1. struct sk_buff {
2. union {
3. struct {
4. /* These two members must be first. */
5. struct sk_buff *next; //指向下一个 sk_buff 结构体
6. struct sk_buff *prev; //指向前一个 sk_buff 结构体
7. ……
8. };
9. ……
10. };
11.
12. ……
13.
14. unsigned int len, //数据包的总长度,包括线性数据和非线性数据
15. data_len; //非线性的数据长度
16. __u16 mac_len, //mac 包头长度
```

```
17. hdr_len;
18.
19. ……
20.
21. __u32 priority; //当前 sk_buff 结构体的优先级
22.
23. ……
24.
25. __be16 protocol; //存放上层的协议类型,可以通过 eth_type_trans()来获
取
26. __u16 transport_header; //传输层头部的偏移值
27. __u16 network_header; //网络层头部的偏移值
28. __u16 mac_header; //MAC 数据链路层头部的偏移值
29.
30. ……
31.
32. /* public: */
33.
34. /* These elements must be at the end, see alloc_skb() for details. */
35. sk_buff_data_t tail; //指向缓冲区的数据包末尾
36. sk_buff_data_t end; //指向缓冲区的末尾
37. unsigned char *head, //指向缓冲区的协议头开始位置
38. *data; //指向缓冲区的数据包开始位置
39. ……
40. };
```

sk_buff 中个数据段主要排布空间如下：

MAC 头 IP 头 数据类型 数据

MAC 头用结构体 ethhdr 表示。

IP 头用结构体 iphdr 表示。

数据类型为一个字节。

发包函数 ndo_start_xmit 中具体要做的是如下：

1) 调用函数 netif_stop_queue()停止上层下传数据。

2) 通过硬件发送数据。

3) 调用函数 dev_kfree_skb()释放 sk_buff。

4) 发送成功进入中断时，更新统计信息，调用 netif_wake_queue()函数，使上层继续下发数据

5) 发送超时则在 net_device_ops 中的 ndo_tx_timeout 函数中调用 netif_wake_queue()函数，使上层继续下发数据。

dev_kfree_skb()函数原型如下：

```
#define dev_kfree_skb(a) consume_skb(a)
void consume_skb(struct sk_buff *skb)
```

netif_wake_queue()函数原型如下：

```
static inline void netif_wake_queue(struct net_device *dev)
```

netif_stop_queue()函数原型如下：

```
static inline void netif_stop_queue(struct net_device *dev)
```

3、收包

收包一般放在中断中处理。处理过程为：

1) 使用 alloc_skb()函数构造一个 sk_buff。

2) 使用 skb_reserve(sk_buff,2)把 sk_buff 里的数据包先后位移 2 字节以腾出 sk_buff 里的头部空间。

3) 读取网络设备硬件上接收到的数据，把数据复制到 sk_buff 的成员 data 指针。

4) 使用 eth_type_trans()函数来获取上层协议，将返回值赋给 sk_buff 的成员 protocol。

5) 更新统计信息，最后使用 netif_rx()将 sk_buff 上传到上层协议。

9.2 网络驱动实验测试

1、网络设备驱动一般包括 SoC 端的 MAC 外设驱动和外接的 PHY 驱动，我们所使用的内核源码，默认已经使能了 MAC 外设驱动模块，接下来我们需要对内核和设备树进行配置，使能 PHY 芯片所对应的驱动模块。

2、网络外设设备树

我们就来简单分析一下设备树。在内核源码目录 arch/arm/boot/dts/zynqmp.dtsi 设备树文件中，定义了四个网络设备节点，如下所示：

```
gem0: ethernet@ff0b0000 {
        compatible = "cdns,zynqmp-gem", "cdns,gem";
        status = "disabled";
        interrupt-parent = <&gic>;
        interrupts = <0 57 4>, <0 57 4>;
        reg = <0x0 0xff0b0000 0x0 0x1000>;
        clock-names = "pclk", "hclk", "tx_clk";
        #address-cells = <1>;
        #size-cells = <0>;
        #stream-id-cells = <1>;
        iommus = <&smmu 0x874>;
        power-domains = <&zynqmp_firmware PD_ETH_0>;
};

gem1: ethernet@ff0c0000 {
        compatible = "cdns,zynqmp-gem", "cdns,gem";
        status = "disabled";
        interrupt-parent = <&gic>;
        interrupts = <0 59 4>, <0 59 4>;
        reg = <0x0 0xff0c0000 0x0 0x1000>;
        clock-names = "pclk", "hclk", "tx_clk";
        #address-cells = <1>;
        #size-cells = <0>;
        #stream-id-cells = <1>;
        iommus = <&smmu 0x875>;
        power-domains = <&zynqmp_firmware PD_ETH_1>;
};

gem2: ethernet@ff0d0000 {
        compatible = "cdns,zynqmp-gem", "cdns,gem";
        status = "disabled";
```

```
                interrupt-parent = <&gic>;
                interrupts = <0 61 4>, <0 61 4>;
                reg = <0x0 0xff0d0000 0x0 0x1000>;
                clock-names = "pclk", "hclk", "tx_clk";
                #address-cells = <1>;
                #size-cells = <0>;
                #stream-id-cells = <1>;
                iommus = <&smmu 0x876>;
                power-domains = <&zynqmp_firmware PD_ETH_2>;
        };

        gem3: ethernet@ff0e0000 {
                compatible = "cdns,zynqmp-gem", "cdns,gem";
                status = "disabled";
                interrupt-parent = <&gic>;
                interrupts = <0 63 4>, <0 63 4>;
                reg = <0x0 0xff0e0000 0x0 0x1000>;
                clock-names = "pclk", "hclk", "tx_clk";
                #address-cells = <1>;
                #size-cells = <0>;
                #stream-id-cells = <1>;
                iommus = <&smmu 0x877>;
                power-domains = <&zynqmp_firmware PD_ETH_3>;
        };
```

四个节点的 status 属性的值都是"disabled"，也就是默认情况它们都是关闭的，根据 vivado 的配置情况，在 pcw.dtsi 设备树文件中使能相应的网络设备节点。

再打开 arch/arm/boot/dts/pcw.dtsi 设备树文件看看，找到已使能节点的追加内容，这些内容都是根据 vivado 配置情况自动生成的：

```
&gem3 {
    local-mac-address = [00 0a 35 00 11 55];
    phy-reset-gpio = <&gpio 44 GPIO_ACTIVE_LOW>;
    phy-reset-active-low;
};
```

phy-mode：该属性的值是一个字符串，表示网络所使用的 PHY 接口模式，是 MII、RMII、GMII 还是 RGMII 等，它是一个标准的属性，支持的字符串值有"mii"、"gmii"、"sgmii"、"rmii"、"rgmii"、"rgmii-id"、"rgmii-rxid"、"rgmii-txid"、"xgmii"等等。

phy-mode 属性的值等于"rgmii-id"表示它使用的是 RGMII 接口模式；

phy-handle：连接到此网络设备的 PHY 芯片句柄。也就是说该属性用于指定网口设备对应的 PHY 设备节点，这个属性是一个必要属性，有些需要我们在 system-top.dts 文件进行修改。

phy-reset-gpio：用于描述 PHY 芯片对应的复位引脚。

phy-reset-active-low：该属性表示 PHY 芯片复位是低电平有效，也就是低电平处于复位状态，高电平结束复位。

local-mac-address：该属性用于指定分配给网络设备的 MAC 地址，它的值是一个 6 个字节的数组，譬如 local-mac-address = [00 0a 35 00 11 55]。

```
Starting tcf-agent: OK

root@rfevm:~#
root@rfevm:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0A:35:00:11:55
          inet addr:192.168.1.10  Bcast:0.0.0.0  Mask:255.255.255.0
          inet6 addr: fe80::20a:35ff:fe00:1155/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:35 errors:0 dropped:0 overruns:0 frame:0
          TX packets:12 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2236 (2.1 KiB)  TX bytes:1128 (1.1 KiB)
          Interrupt:36

root@rfevm:~# ping 192.168.1.12
PING 192.168.1.12 (192.168.1.12): 56 data bytes
64 bytes from 192.168.1.12: seq=0 ttl=128 time=0.275 ms
64 bytes from 192.168.1.12: seq=1 ttl=128 time=0.235 ms
64 bytes from 192.168.1.12: seq=2 ttl=128 time=0.190 ms
^C
--- 192.168.1.12 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.190/0.233/0.275 ms
root@rfevm:~#
```

9.3 外网连接测试

1、我们使用的根文件系统，默认情况下打开了 eth0 网络设备，如果我们在启动开发板之前，需要将网口连接到路由器。

2、设备启动之后进入到命令行，输入命令自动获取 IP 地址：

udhcpc

```
root@rfevm:~# udhcpc
udhcpc: started, v1.32.0
udhcpc: sending discover
udhcpc: sending select for 192.168.5.192
udhcpc: lease of 192.168.5.192 obtained, lease time 86400
/etc/udhcpc.d/50default: Adding DNS 192.168.5.1
root@rfevm:~#
```

接下来我们需要测试下网口的联网功能是否正常，也就是测试它们是否能够上网，怎么测呢？一般通过 ping 命令即可，现在开发板 eth0 已经打开，并且已经自动获取到了 IP 地址，执行 ping 命令可以测试 eth0 的联网功能是否正常，这里我们以百度网站为例：

```
root@rfevm:~# ping www.baidu.com
PING www.baidu.com (14.119.104.254): 56 data bytes
64 bytes from 14.119.104.254: seq=0 ttl=55 time=37.274 ms
64 bytes from 14.119.104.254: seq=1 ttl=55 time=36.946 ms
64 bytes from 14.119.104.254: seq=2 ttl=55 time=36.462 ms
64 bytes from 14.119.104.254: seq=3 ttl=55 time=35.619 ms
64 bytes from 14.119.104.254: seq=4 ttl=55 time=36.314 ms
64 bytes from 14.119.104.254: seq=5 ttl=55 time=36.265 ms
64 bytes from 14.119.104.254: seq=6 ttl=55 time=36.198 ms
64 bytes from 14.119.104.254: seq=7 ttl=55 time=36.181 ms
64 bytes from 14.119.104.254: seq=8 ttl=55 time=36.494 ms
```

9.4 以太网网速测试

1、前面就给大家介绍过了，网口都是千兆以太网接口，理论最大通信速率可达 1000mb/s，那么本小节我们就来测试下这两个网口的实际速率，测试过程我们使用开发板和 PC 机直连的方式。

2、Linux 下网络速率测试可以使用 iper 工具，iperf 是一个网络性能测试工具，可以测试 TCP 和 UDP 的带宽质量，测量 TCP 最大带宽，具有多种参数和 UDP 特性，报告带宽、延迟抖动和数据丢包等；这里我们仅仅只是使用 iperf 工具测试开发板上

两个网口的上下行（上传和下载）带宽（就是速率），其它的一些参数特性就不给大家进行详细测试了。iperf 的测试方法需要有一台主机作为服务器端，另一台主机作为客户端，客户端向服务器端发送数据，这里我们可以将主机作为服务器端，而开发板作为客户端进行测试。

这个测试过程我们需要用到 iperf 工具，但是我们当前使用的根文件系统当中并没有提供这个工具，所以我们需要自己移植到开发板中。

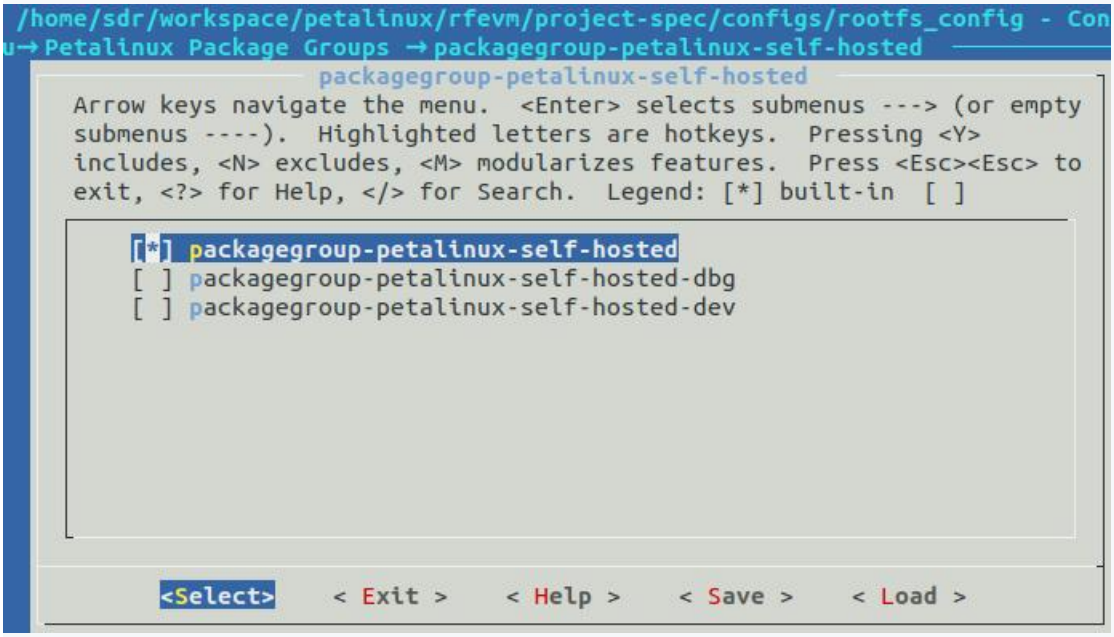
3、添加 gcc 编译工具链

要想在开发板上直接编译 C/C++源码，可以添加 gcc 编译工具链 ，也就是 packagegroup-petalinux-self-hosted，self-hosted 直译就是自主机的，也就是把开发板当作主机来开发使用。添加 packagegroup-petalinux-self-hosted 需要进行如下配置：

Petalinux Package Groups --->

packagegroup-petalinux-self-hosted --->

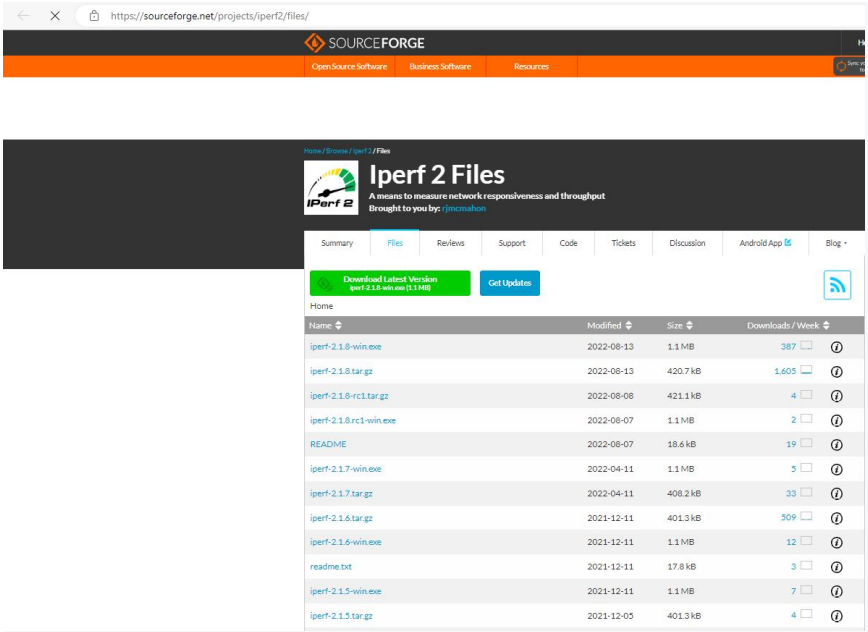
packagegroup-petalinux-self-hosted（Y）



packagegroup-self-hosted 中包含着 gcc 编译工具链；注意该包是可选的，添加会增大根文件系统的大小，需要的话可以添加。

4、下载安装 iperf

下载地址：https://sourceforge.net/projects/iperf2/files/



下载这个版本：

https://udomain.dl.sourceforge.net/project/iperf2/iperf-2.1.8.tar.gz

将下载的文件导入设备里面，然后执行以下命令：

```
tar -xvf iperf-2.1.8.tar.gz
cd iperf-2.1.8/
mkdir _install
./configure --host=aarch64-xilinx-linux --prefix=$(pwd)/_install
make
make install
```

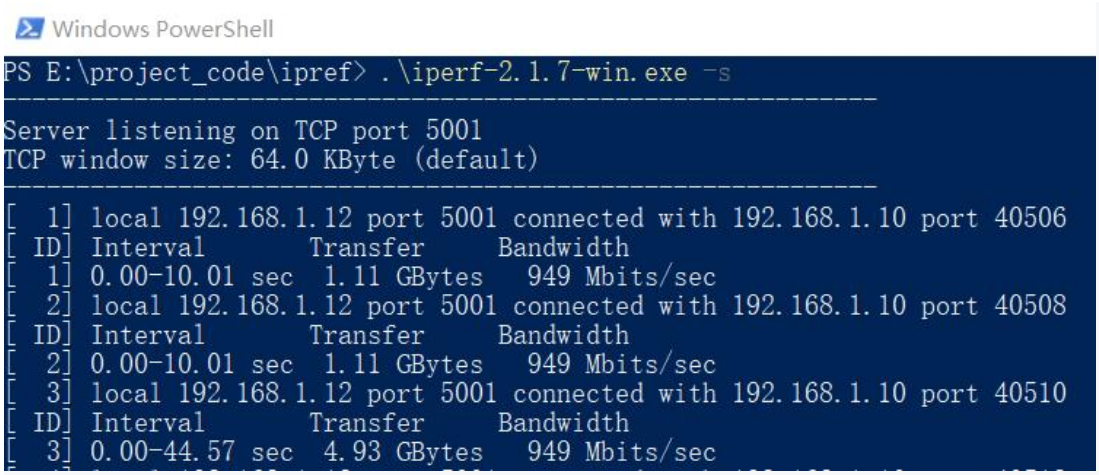
```
root@rfevm:/media/card0/iperf-2.1.8# make install
Making install in compat
make[1]: Entering directory '/media/card0/iperf-2.1.8/compat'
make[2]: Entering directory '/media/card0/iperf-2.1.8/compat'
make[2]: Nothing to be done for 'install-exec-am'.
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/media/card0/iperf-2.1.8/compat'
make[1]: Leaving directory '/media/card0/iperf-2.1.8/compat'
Making install in doc
make[1]: Entering directory '/media/card0/iperf-2.1.8/doc'
make[2]: Entering directory '/media/card0/iperf-2.1.8/doc'
make[2]: Nothing to be done for 'install-exec-am'.
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/media/card0/iperf-2.1.8/doc'
make[1]: Leaving directory '/media/card0/iperf-2.1.8/doc'
Making install in include
make[1]: Entering directory '/media/card0/iperf-2.1.8/include'
make[2]: Entering directory '/media/card0/iperf-2.1.8/include'
make[2]: Nothing to be done for 'install-exec-am'.
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/media/card0/iperf-2.1.8/include'
make[1]: Leaving directory '/media/card0/iperf-2.1.8/include'
Making install in src
make[1]: Entering directory '/media/card0/iperf-2.1.8/src'
make[2]: Entering directory '/media/card0/iperf-2.1.8/src'
 /bin/mkdir -p '/media/card0/iperf-2.1.8/_install/bin'
  /usr/bin/install -c iperf '/media/card0/iperf-2.1.8/_install/bin'
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/media/card0/iperf-2.1.8/src'
make[1]: Leaving directory '/media/card0/iperf-2.1.8/src'
Making install in man
make[1]: Entering directory '/media/card0/iperf-2.1.8/man'
make[2]: Entering directory '/media/card0/iperf-2.1.8/man'
make[2]: Nothing to be done for 'install-exec-am'.
 /bin/mkdir -p '/media/card0/iperf-2.1.8/_install/share/man/man1'
 /usr/bin/install -c -m 644 iperf.1 '/media/card0/iperf-2.1.8/_install/share/man/man1'
make[2]: Leaving directory '/media/card0/iperf-2.1.8/man'
make[1]: Leaving directory '/media/card0/iperf-2.1.8/man'
make[1]: Entering directory '/media/card0/iperf-2.1.8'
make[2]: Entering directory '/media/card0/iperf-2.1.8'
make[2]: Nothing to be done for 'install-exec-am'.
make[2]: Nothing to be done for 'install-data-am'.
make[2]: Leaving directory '/media/card0/iperf-2.1.8'
make[1]: Leaving directory '/media/card0/iperf-2.1.8'
root@rfevm:/media/card0/iperf-2.1.8#
root@rfevm:/media/card0/iperf-2.1.8#
```

5、测试

（1）在本测试当中，我们将主机作为服务器，而开发板作为客户端；

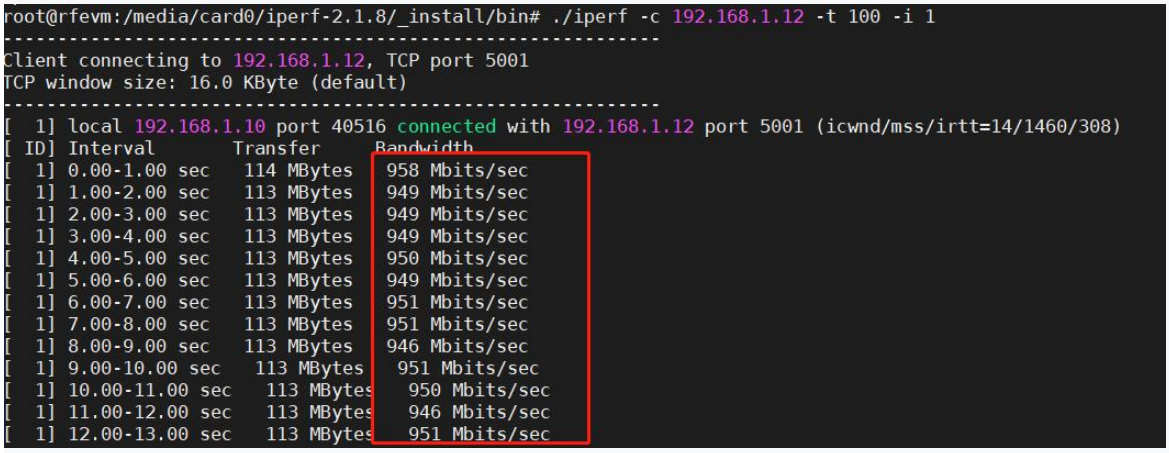
在主机打开 Powershell 执行下面这条命令，将其作为服务器，如下所示：

iperf3 -s



接下来我们需要在开发板终端执行下面这条命令，将开发板作为客户端，并进行测试：

./iperf -c 192.168.1.12 -t 100 -i 1



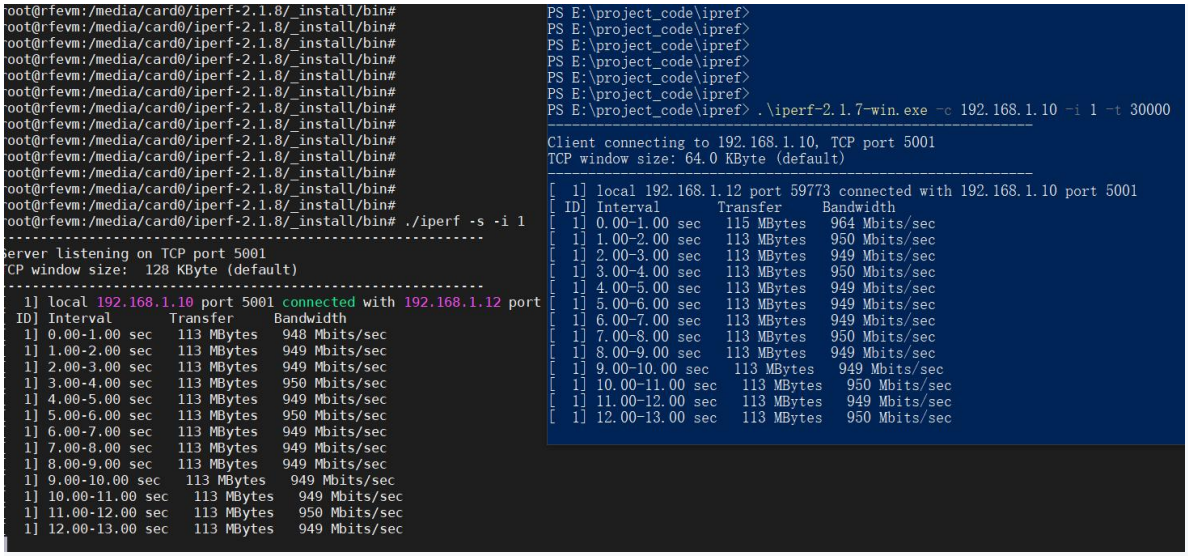
（2）在本测试当中，我们将开发板作为服务器，而主机作为客户端；

在开发板终端执行下面这条命令，将开发板作为服务器：

./iperf -c 192.168.1.12 -t 100 -i 1

在主机打开 Powershell 执行下面这条命令，将其作为客户端，如下所示：

.\iperf-2.1.7-win.exe -c 192.168.1.10 -i 1 -t 30000



由此可以知道开发板板上的 PS 网口是千兆以太网接口。